



UNIVERSITAS INDONESIA

JUDUL SKRIPSI/THESIS/DISERTASI

SKRIPSI

MOHAMMAD RAFFY ZEIDAN

2206051462

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

DEPOK

2026



UNIVERSITAS INDONESIA

JUDUL SKRIPSI/THESIS/DISERTASI

SKRIPSI

**Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Statistika**

MOHAMMAD RAFFY ZEIDAN

2206051462

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

DEPOK

MEI 2026

HALAMAN PERSETUJUAN

Judul : Judul Skripsi/Thesis/Disertasi
Nama : Mohammad Raffy Zeidan
NPM : 2206051462

Laporan Skripsi ini telah diperiksa dan disetujui.

XX Juli 2019

Prof. ???
Pembimbing Skripsi

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Tanda Tangan :

Tanggal : XX Juli 2019

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :
Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Program Studi : Statistika
Judul Skripsi : Judul Skripsi/Thesis/Disertasi

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing : Prof. ??? ()

Penguji : Prof. XXX ()

Penguji : Prof. XXXX ()

Penguji : Prof. XXXXXX ()

@todo

Jangan lupa mengisi nama para penguji.

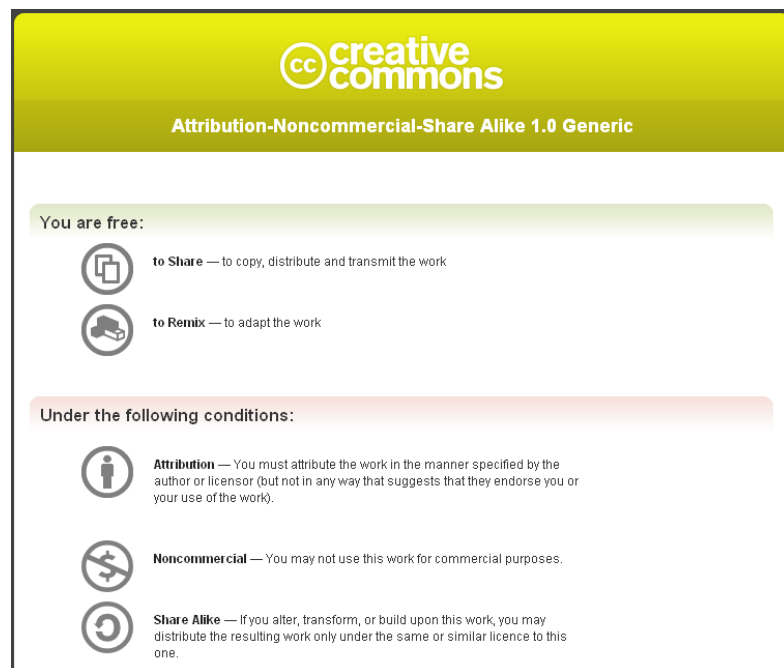
Ditetapkan di : Depok
Tanggal : XX Juli 2019

KATA PENGANTAR

Template ini disediakan untuk orang-orang yang berencana menggunakan $\text{\LaTeX}$ untuk membuat dokumen tugas akhirnya. Mengapa $\text{\LaTeX}$? Ada banyak hal mengapa menggunakan $\text{\LaTeX}$, diantaranya:

1. $\text{\LaTeX}$ membuat kita jadi lebih fokus terhadap isi dokumen, bukan tampilan atau halaman.
2. $\text{\LaTeX}$ memudahkan dalam penulisan persamaan matematis.
3. Adanya otomatis dalam penomoran caption, bab, subbab, subsubbab, referensi, dan rumus.
4. Adanya otomatisasi dalam pembuatan daftar isi, daftar gambar, dan daftar tabel.
5. Adanya kemudahan dalam memberikan referensi dalam tulisan dengan menggunakan label. Cara ini dapat meminimalkan kesalahan pemberian referensi.

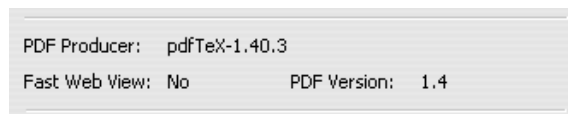
Template ini bebas digunakan dan didistribusikan sesuai dengan aturan *Creative Common License 1.0 Generic*, yang secara sederhana berisi:



Gambar 1: *Creative Common License 1.0 Generic*

Gambar 1 diambil dari http://creativecommons.org/licenses/by-nc-sa/1.0/deed.en_CA. Jika ingin mengetahui lebih lengkap mengenai *Creative Common License 1.0 Generic*, silahkan buka <http://creativecommons.org/licenses/by-nc-sa/1.0/legalcode>. Seluruh dokumen yang dibuat dengan menggunakan template ini sepenuhnya menjadi hak milik pembuat dokumen dan bebas didistribusikan sesuai dengan keperluan masing-masing. Lisensi hanya berlaku jika ada orang yang membuat template baru dengan menggunakan template ini sebagai dasarnya.

Dokumen ini dibuat dengan $\text{\LaTeX}$ juga. Untuk meyakinkan Anda, coba lihat properti dari dokumen ini dan Anda akan menemukan bagian seperti Gambar 2. Dokumen ini dimaksudkan untuk memberikan gambaran kepada Anda seperti apa mudahnya menggunakan $\text{\LaTeX}$ dan juga memperlihatkan betapa bagus dokumen yang dihasilkan. Seluruh url yang Anda temukan dapat Anda klik. Seluruh referensi yang ada juga dapat diklik. Untuk mengerti template yang disediakan, Anda tetap harus membuka kode $\text{\LaTeX}$ dan bermain-main dengannya. Penjelasan dalam PDF ini masih bersifat gambaran dan tidak begitu mendetail, dapat dianggap sebagai pengantar singkat. Jika Anda merasa kesulitan dengan template ini, mungkin ada baiknya Anda belajar sedikit dasar-dasar $\text{\LaTeX}$.



Gambar 2: Dokumen Dibuat dengan PDF $\text{\LaTeX}$

Semoga template ini dapat membantu orang-orang yang ingin mencoba menggunakan $\text{\LaTeX}$. Semoga template ini juga tidak berhenti disini dengan ada kontribusi dari para penggunanya. Kami juga ingin berterima kasih kepada Andreas Febrian, Lia Sadita, Fahrurrozi Rahman, Andre Tampubolon, dan Erik Dominikus atas kontribusinya dalam template ini.

Depok, 27 Februari 2019

Mohammad Raffy Zeidan

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Program Studi : Statistika
Fakultas : Matematika dan Ilmu Pengetahuan
Alam
Jenis Karya : Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Judul Skripsi/Thesis/Disertasi

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : XX Juli 2019
Yang menyatakan

(Mohammad Raffy Zeidan)

ABSTRAK

Nama : Mohammad Raffy Zeidan
Program Studi : Statistika
Judul : Judul Skripsi/Thesis/Disertasi
Pembimbing : Prof. ???

Tesis ini membahas kemampuan mahasiswa Fakultas Psikologi UI dalam mencari dan menggunakan informasi secara efektif dalam konteks *active learning* dan *self regulated learning* selama mereka mengikuti Program Pendidikan Dasar Pendidikan Tinggi. Penelitian ini adalah penelitian kualitatif dengan desain deskriptif. Hasil penelitian menyarankan bahwa perpustakaan perlu dilibatkan dalam pengembangan kurikulum; materi pendidikan pemakai perpustakaan harus dikembangkan sesuai dengan komponen-komponen yang ada dalam *information literacy*; perpustakaan juga harus menyediakan sarana dan fasilitas yang mendukung peningkatan *literacy* mahasiswa.

Kata kunci:

Informasi, *information literacy*, *information skills*

ABSTRACT

Name : Mohammad Raffy Zeidan
Study Program : Statistika
Title : Unknown Title for Final Report/Thesis/Disertation
Counsellor : Prof. ???

The focus of this study is the freshman student of Faculty of Psychology at University of Indonesia experience of acquiring, evaluating and using information, when they enroll in “Program Dasar Pendidikan Tinggi (PDPT)”. The purpose of this study is to understand how freshman students acquire, evaluate and use information. Knowing this will allow library to identify changes should be made to improve user education program at University of Indonesia. This research is qualitative descriptive interpretive. The data were collected by means of deep interview. The researcher suggests that library should improve the user education program and provide facilities which can help students to be information literate.

Key words:

Information literacy, information skills, information

DAFTAR ISI

HALAMAN JUDUL	i
LEMBAR PERSETUJUAN	ii
LEMBAR PERNYATAAN ORISINALITAS	iii
LEMBAR PENGESAHAN	iv
KATA PENGANTAR	v
LEMBAR PERSETUJUAN PUBLIKASI ILMIAH	vi
ABSTRAK	viii
DAFTAR ISI	x
DAFTAR GAMBAR	xiii
DAFTAR TABEL	xiv
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Permasalahan	5
1.3 Tujuan Penelitian	6
1.4 Manfaat Penelitian	6
1.5 Ruang Lingkup Penelitian	7
2 LANDASAN TEORI	8
2.1 Analisis Sentimen	8
2.2 Pendeteksian Topik	9
2.3 Neural Network	9
2.3.1 Fungsi Aktivasi	11
2.3.1.1 <i>Softmax</i>	11
2.3.1.2 <i>Rectified Linear Unit</i> (ReLU)	12
2.3.1.3 <i>Gaussian Error Linear Unit</i> (GELU)	12
2.3.2 Fungsi <i>Loss</i>	13

2.3.3	Backpropagation	13
2.3.4	Teknik Optimasi	15
2.3.4.1	<i>Stochastic Gradient Descent</i> (SGD)	15
2.3.4.2	<i>Adaptive Moment Estimation</i> (Adam)	15
2.3.5	Regularisasi	16
2.3.6	Metrik Evaluasi	17
2.4	Autoencoder	18
2.4.1	Encoder dan Decoder	18
2.4.2	Fungsi Objektif	19
2.5	Bidirectional Encoder Representations from Transformers (BERT) .	19
2.5.1	Arsitektur BERT	20
2.5.2	Mekanisme <i>Self-Attention</i>	20
2.5.3	Representasi Input BERT	21
2.6	Multi-Task Learning	22
2.6.1	Formulasi MTL	23
2.6.2	Hard Parameter Sharing	24
2.6.3	Soft Parameter Sharing	25
2.7	Clustering	25
2.7.1	K-Means Clustering	26
2.8	Interpretabilitas Model <i>Deep Learning</i>	27
2.8.1	<i>Occlusion-based Token Attribution</i>	28
2.8.2	<i>Integrated Gradients</i>	29

3 METODE MODEL JOINT SENTIMENT-TOPIC BERBASIS MULTITASK LEARNING

30

3.1	<i>Bidirectional Encoder Representations from Transformers</i> (BERT) Sebagai Model Representasi Teks	30
3.2	Alur Pelatihan Model <i>Multitask</i>	33
3.2.1	Implementasi <i>Autoencoder</i>	33
3.2.2	<i>Deep Embedded Clustering</i> (DEC)	34
3.2.3	Pelatihan <i>Multitask</i> : JST-DEC	36
3.2.3.1	<i>Clustering Head</i>	36
3.2.3.2	Analisis Sentimen Menggunakan <i>Sentiment Head</i>	37
3.2.3.3	Pelatihan <i>Joint</i> dengan <i>Multitask Loss</i>	38
3.2.3.4	Algoritma JST-DEC	38
3.3	Interpretasi Model dengan Analisis Atribusi Token	39
3.3.1	Implementasi <i>Occlusion-based Token Attribution</i> dan Agre- gasi per Kluster	39

3.3.2	Implementasi <i>Integrated Gradients</i>	40
3.4	Evaluasi Topik	41
3.4.1	<i>Topic Coherence</i> Berbasis NPMI	41
3.4.2	<i>Topic Diversity</i>	42
4	HASIL DAN SIMULASI DAN PEMBAHASAN	44
4.1	Spesifikasi Mesin dan Perangkat Lunak	44
4.2	Tahapan Umum Simulasi	44
4.3	Data	46
4.3.1	Pengumpulan Data	46
4.3.2	Prapengolahan Data	47
4.3.3	Representasi Teks	49
4.3.4	Pembagian Data	49
4.4	Implementasi Model	50
4.4.1	Konfigurasi JST-DEC	50
4.4.2	Konfigurasi <i>Joint Training</i>	51
4.4.3	Konfigurasi Model Pembandingan	52
4.4.3.1	NN-BERT	52
4.4.3.2	BERTopic	53
4.5	Analisis Kinerja Model	54
4.5.1	Analisis Kinerja Pra-pelatihan	54
4.5.2	Analisis Kinerja <i>Joint Training</i>	55
4.5.2.1	Hasil Evaluasi pada Dataset IMDB	55
4.5.2.2	Hasil Evaluasi pada Dataset Yelp	55
5	KESIMPULAN	56
5.1	Kesimpulan	56
5.2	Saran	56

DAFTAR GAMBAR

Gambar 1	<i>Creative Common License 1.0 Generic</i>	v
Gambar 2	Dokumen Dibuat dengan PDFLatex	vi
Gambar 2.1	Ilustrasi <i>neural network</i> secara biologis (Aggarwal, 2018) .	10
Gambar 2.2	<i>Artificial Neural Network</i> dengan representasi bobot sinapsis (Aggarwal, 2018)	10
Gambar 2.3	Arsitektur <i>feed-forward neural network</i> (Aggarwal, 2018) .	11
Gambar 2.4	Grafik fungsi aktivasi: <i>Softmax</i> , ReLU, dan GELU	13
Gambar 2.5	Ilustrasi graf komputasi <i>multi-layer perceptron</i> (Aggarwal, 2018)	14
Gambar 2.6	Ilustrasi <i>bias-variance tradeoff</i> terhadap kompleksitas model (Aggarwal, 2018, telah diolah kembali)	16
Gambar 2.7	Arsitektur <i>autoencoder</i> menunjukkan proses <i>encoding</i> dan <i>decoding</i> (Aggarwal, 2018)	18
Gambar 2.8	Arsitektur BERT dengan beberapa lapisan <i>Transformer Encoder</i> (Devlin <i>et al.</i> , 2018)	20
Gambar 2.9	Representasi <i>input</i> BERT yang terdiri dari <i>token</i> , <i>segment</i> , dan <i>position embedding</i> (Devlin <i>et al.</i> , 2018)	22
Gambar 2.10	Ilustrasi arsitektur <i>hard parameter sharing</i> dalam <i>Multi-Task Learning</i> (Ruder, 2017, telah diolah kembali)	24
Gambar 2.11	Ilustrasi arsitektur <i>soft parameter sharing</i> (Ruder, 2017, telah diolah kembali)	25
Gambar 3.1	Alur metode analisis sentimen dan pendeteksian topik	30
Gambar 3.2	Representasi teks menggunakan BERT ((Devlin <i>et al.</i> , 2018); telah diolah kembali).	31
Gambar 3.3	Ilustrasi <i>autoencoder</i> lengkap (kiri) dan komponen <i>encoder</i> setelah <i>decoder</i> dilepas (kanan).	34
Gambar 3.4	Arsitektur <i>clustering head</i> berbasis DEC.	37
Gambar 3.5	Arsitektur <i>sentiment head</i>	37
Gambar 4.1	Diagram alur simulasi penelitian.	45

DAFTAR TABEL

Tabel 2.1	Algoritma K-Means <i>Clustering</i>	27
Tabel 3.1	Algoritma <i>Deep Embedded Clustering</i> (DEC)	36
Tabel 3.2	Algoritma JST-DEC (<i>Joint Sentiment-Topic Deep Embedded Clustering</i>)	39
Tabel 4.1	Spesifikasi mesin dan perangkat lunak	44
Tabel 4.2	Cuplikan data ulasan IMDB (Maas <i>et al.</i> , 2011b).	47
Tabel 4.3	Cuplikan data ulasan Yelp (X. Zhang, J. Zhao, dan LeCun, 2015b).	47
Tabel 4.4	Contoh hasil prapengolahan pada dataset IMDB dan Yelp.	48
Tabel 4.5	Contoh proses tokenisasi menggunakan WordPiece BERT.	49
Tabel 4.6	Contoh representasi vektor [CLS] hasil ekstraksi BERT.	49
Tabel 4.7	Konfigurasi pra-pelatihan <i>autoencoder</i>	50
Tabel 4.8	Konfigurasi komponen DEC	51
Tabel 4.9	Konfigurasi <i>joint training</i> JST-DEC	52
Tabel 4.10	Konfigurasi model NN-BERT	53
Tabel 4.11	Konfigurasi model BERTopic	53
Tabel 4.12	Perkembangan <i>loss</i> pra-pelatihan pada dataset IMDB	54
Tabel 4.13	Perkembangan <i>loss</i> pra-pelatihan pada dataset Yelp	54
Tabel 4.14	Hasil evaluasi model pada dataset IMDB	55
Tabel 4.15	Hasil evaluasi model pada dataset Yelp	55

DAFTAR KODE

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dunia kontemporer dibanjiri oleh arus deras informasi tekstual yang dihasilkan setiap saat, bersumber dari platform media sosial, ulasan daring, artikel berita, repositori ilmiah, dan banyak lagi. Kumpulan data tekstual masif ini, yang sebagian besar tidak terstruktur, mengandung potensi pengetahuan yang signifikan mengenai sentimen masyarakat, dinamika pasar, hingga tren diskursus publik. Akan tetapi, volume dan keragaman data tersebut menjadikan analisis manual suatu hal yang tidak praktis dan efisien. Menjawab tantangan ini, disiplin *Natural Language Processing* (NLP), sebagai bagian dari kecerdasan artifisial yang menjembatani komunikasi antara mesin dan bahasa alami manusia, hadir menawarkan metodologi komputasional untuk mengolah dan memahami data linguistik dalam skala besar.

Dalam spektrum kapabilitas NLP, dua teknik yang memiliki relevansi tinggi untuk mengekstrak makna dari himpunan data tekstual adalah analisis sentimen (*Sentiment Analysis*) dan pemodelan topik (*Topic Modeling*) (Blei, Ng, dan Jordan, 2003). Analisis sentimen secara khusus dirancang untuk menyelidiki dan menentukan muatan subjektif atau nada emosional yang terkandung dalam suatu teks, lalu menggolongkannya sebagai ekspresi positif, negatif, ataupun netral. Pengaplikasian teknik ini sangat berharga untuk melacak citra merek, menganalisis tanggapan konsumen, memetakan opini publik terkait isu tertentu, serta mengidentifikasi ujaran problematik. Kemampuan mengukur sentimen secara kolektif memberikan landasan bagi entitas bisnis maupun riset untuk memahami persepsi dan mengambil langkah strategis.

Di sisi lain, pemodelan topik menangani tugas komplementer, yakni mengungkap tema-tema atau pokok bahasan esensial yang tersembunyi (*latent thematic structures*) di dalam koleksi dokumen berjumlah besar, atau sering disebut *corpora* (Blei, Ng, dan Jordan, 2003). Berbeda dengan fokus pada nuansa afektif per teks, tujuan utama pemodelan topik adalah menyajikan intisari tematik dari keseluruhan data. Kegunaannya terbentang luas, mulai dari fasilitasi eksplorasi dan pengorganisasian arsip digital, identifikasi pola baru dalam korpus literatur atau berita, hingga penyempurnaan sistem rekomendasi konten dan pemahaman struktur narasi dalam

skala makro. Dengan mengabstraksikan topik-topik dominan, metode ini membantu mereduksi kompleksitas informasi dan menyajikan gambaran umum konten. Walaupun memiliki fokus yang berbeda, kedua pendekatan ini berpotensi menghasilkan pemahaman yang lebih komprehensif jika diintegrasikan, menyingkap *apa* subjek pembicaraan sekaligus *bagaimana* pandangan atau perasaan terhadap subjek tersebut.

Pendekatan tradisional dalam pemodelan topik, seperti *Latent Dirichlet Allocation* (LDA), umumnya berbasis probabilistik dan mengandalkan metode faktorisasi matriks untuk mengidentifikasi distribusi kata dalam topik (Blei, Ng, dan Jordan, 2003; Blei, 2012). Meskipun metode ini telah terbukti efektif dalam banyak kasus, keterbatasannya dalam menangkap hubungan semantik yang kompleks dalam teks membuatnya kurang optimal untuk tugas yang lebih kompleks, seperti klasifikasi sentimen yang dilakukan secara simultan dengan deteksi topik. Salah satu kelemahan utama pendekatan probabilistik seperti LDA adalah skalabilitasnya yang terbatas, terutama saat diaplikasikan pada *dataset* berukuran besar dengan dimensi fitur tinggi (Boyd-Graber, Hu, dan Mimno, 2017). Selain itu, model berbasis probabilistik sering kali memerlukan waktu komputasi yang signifikan dan sulit diadaptasi untuk menangani data teks modern yang bersifat dinamis dan tidak terstruktur (McAuliffe dan Blei, 2008).

Dalam klasifikasi sentimen, pendekatan berbasis pembelajaran mendalam (*deep learning*) telah menunjukkan keunggulan signifikan dibandingkan metode tradisional. Salah satu model yang paling efektif adalah *Bidirectional Encoder Representations from Transformers* (BERT), yang mampu memahami konteks kata dalam teks dengan mempertimbangkan hubungan antar kata di kedua arah (*bidirectional*) (Devlin *et al.*, 2018). BERT telah banyak digunakan dalam tugas-tugas NLP dan terbukti unggul dalam klasifikasi sentimen, menghasilkan akurasi yang lebih tinggi dibandingkan pendekatan berbasis fitur atau metode statistik konvensional (Sun *et al.*, 2019).

Seiring dengan berkembangnya model *deep learning*, pendekatan berbasis jaringan saraf telah terbukti lebih unggul dalam menangani data teks yang tidak terstruktur dibandingkan metode berbasis probabilistik seperti LDA (Mikolov, K. Chen *et al.*, 2013). Model *neural network* dapat mengekstrak representasi semantik yang lebih kaya dan menangkap hubungan kontekstual antar kata dalam teks dengan lebih baik (Yinhan Zhang, H. Zhao, dan M. Li, 2020). Selain itu, pendekatan *neural network* dapat mengatasi keterbatasan skalabilitas pada metode probabilistik

karena dapat memanfaatkan arsitektur yang lebih efisien seperti *transformers* dan teknik *embedding* untuk mengurangi dimensi data (Vaswani *et al.*, 2017).

Salah satu pendekatan berbasis *neural network* yang semakin populer dalam pemodelan topik adalah *BERTopic*. Berbeda dengan LDA yang hanya mengandalkan distribusi kata untuk menentukan topik, *BERTopic* memanfaatkan representasi teks dari model *transformer* seperti BERT untuk menangkap hubungan semantik yang lebih dalam antar kata (Grootendorst, 2022). Metode ini menggabungkan *embedding* berbasis *transformer* dengan teknik klusterisasi seperti *UMAP* dan *HDBSCAN* untuk mengidentifikasi topik secara lebih akurat dan stabil, bahkan dalam *dataset* berukuran besar atau dengan struktur yang kompleks. Keunggulan utama *BERTopic* dibandingkan LDA adalah kemampuannya dalam menangani variasi bahasa yang lebih luas, memahami sinonim dan kontekstualisasi kata, serta tidak bergantung pada asumsi distribusi kata seperti pada metode probabilistik (Grootendorst, 2022).

Namun demikian, sebagian besar metode pemodelan topik modern seperti *BERTopic* masih berfokus pada pemisahan topik semata tanpa mempertimbangkan aspek lain seperti sentimen atau struktur laten yang lebih dalam secara eksplisit. Selain itu, integrasi antara pemodelan topik dan klasifikasi sentimen masih sering dilakukan secara terpisah, sehingga informasi semantik yang berharga dalam teks belum sepenuhnya dimanfaatkan secara terpadu. Pemisahan ini mengakibatkan redundansi komputasi dan hilangnya peluang untuk memanfaatkan sinergi antara kedua tugas dalam satu arsitektur model yang kohesif.

Dalam konteks pembelajaran representasi untuk data teks berdimensi tinggi, *Autoencoder* telah menunjukkan kemampuannya sebagai teknik pembelajaran tanpa supervisi (*unsupervised learning*) yang efektif untuk reduksi dimensi dan ekstraksi fitur (Hinton dan Salakhutdinov, 2006). *Autoencoder* bekerja dengan cara mempelajari representasi laten (*latent representation*) yang lebih kompak dari data masukan melalui proses *encoding* dan *decoding*. Arsitektur ini memaksa model untuk mengekstrak informasi paling esensial dari data dengan mengompresi masukan ke dalam ruang dimensi yang lebih rendah, kemudian merekonstruksi kembali data asli dari representasi tersebut. Proses ini memungkinkan model untuk menangkap struktur intrinsik dan pola-pola penting dalam data teks, sekaligus mengurangi *noise* dan redundansi fitur.

Berbeda dengan teknik reduksi dimensi tradisional seperti *Principal Component Analysis* (PCA) yang bersifat linear, *Autoencoder* mampu menangkap hubungan non-linear yang kompleks dalam data (Hinton dan Salakhutdinov, 2006). Hal ini

sangat penting dalam pemrosesan teks, di mana hubungan antar kata dan dokumen seringkali bersifat non-linear dan kontekstual. *Autoencoder* juga memiliki fleksibilitas arsitektur yang memungkinkan penyesuaian kedalaman jaringan sesuai dengan kompleksitas data dan tugas yang dihadapi. Dalam konteks NLP, penggunaan *Autoencoder* telah terbukti efektif untuk berbagai aplikasi, termasuk pembelajaran representasi dokumen, deteksi anomali dalam teks, dan ekstraksi fitur untuk tugas-tugas *downstream* seperti klasifikasi dan klusterisasi (Vincent *et al.*, 2010).

Untuk menjawab keterbatasan integrasi antara pemodelan topik dan klasifikasi sentimen, penelitian ini bertujuan untuk mengembangkan pendekatan pemodelan topik yang lebih terintegrasi melalui kombinasi metode representasi laten berbasis *Autoencoder* dan teknik *Deep Embedded Clustering* (DEC). Dengan memanfaatkan *Autoencoder* sebagai komponen *encoder* untuk menghasilkan representasi teks yang lebih efisien dan bermakna, serta DEC untuk melakukan klusterisasi yang mempertahankan struktur semantik dalam ruang laten, model diharapkan mampu mengidentifikasi topik-topik secara lebih akurat dan representatif. *Autoencoder* dalam penelitian ini berfungsi sebagai fondasi untuk pembelajaran representasi yang *robust*, yang kemudian dioptimalkan lebih lanjut melalui proses klusterisasi DEC untuk menghasilkan pembagian topik yang lebih koheren dan terpisah dengan baik.

Lebih lanjut, penelitian ini mengusulkan pengembangan arsitektur *multi-task learning* (MTL) yang memungkinkan integrasi klasifikasi sentimen dan pemodelan topik secara bersamaan dalam satu kerangka kerja. Pendekatan MTL memiliki keunggulan dalam memanfaatkan hubungan implisit antara tugas-tugas terkait untuk meningkatkan generalisasi model (Caruana, 1997). Dalam konteks penelitian ini, klasifikasi sentimen dan pemodelan topik memiliki keterkaitan semantik yang kuat; sentimen suatu teks seringkali berkorelasi dengan topik yang dibahas, dan pemahaman topik dapat membantu dalam identifikasi sentimen yang lebih akurat. Dengan berbagi representasi laten yang sama melalui *Autoencoder*, kedua tugas dapat saling memperkaya dan menghasilkan performa yang lebih baik dibandingkan jika dilatih secara terpisah.

Pendekatan ini diharapkan dapat meningkatkan efisiensi dan efektivitas analisis teks dengan memanfaatkan representasi laten yang sama untuk dua tugas sekaligus, mengurangi redundansi komputasi, serta memberikan kontribusi terhadap pengembangan model-model NLP yang lebih adaptif dan skalabel. Dengan mengintegrasikan *Autoencoder* untuk pembelajaran representasi, DEC untuk klusterisasi

topik, dan arsitektur MTL untuk klasifikasi sentimen simultan, penelitian ini berupaya menghadirkan solusi komprehensif yang mengatasi keterbatasan pendekatan-pendekatan sebelumnya dalam pemrosesan bahasa alami.

1.2 Rumusan Permasalahan

Dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP), klasifikasi sentimen dan pemodelan topik merupakan dua tugas utama yang sering dilakukan secara terpisah. Pendekatan tradisional seperti *Latent Dirichlet Allocation* (LDA) berbasis probabilistik memiliki keterbatasan dalam menangkap hubungan semantik yang kompleks, sehingga kurang optimal untuk tugas yang membutuhkan pemahaman konteks yang lebih dalam.

Selain itu, pemisahan antara tugas klasifikasi sentimen dan pemodelan topik menyebabkan hilangnya potensi peningkatan performa yang bisa diperoleh dengan memanfaatkan hubungan alami antar tugas tersebut. Model berbasis *deep learning*, seperti *Bidirectional Encoder Representations from Transformers* (BERT), telah terbukti unggul dalam tugas klasifikasi sentimen, namun pendekatan ini belum sepenuhnya diintegrasikan dengan pemodelan topik dalam satu *framework* yang efisien.

Beberapa permasalahan utama yang ingin dijawab dalam penelitian ini adalah:

1. Bagaimana mengembangkan model berbasis *Fully Neural Network* yang dapat menangani klasifikasi sentimen dan pemodelan topik secara simultan?
2. Bagaimana pengaruh pendekatan *Multi-Task Learning* (MTL) terhadap performa model dibandingkan dengan model *single-task*?
3. Bagaimana penerapan *Autoencoder* dapat membantu mengatasi permasalahan dimensi tinggi dalam data teks?
4. Sejauh mana model yang dikembangkan dapat meningkatkan efisiensi dan akurasi dibandingkan dengan metode sebelumnya, seperti analisis sentimen menggunakan EFCM dan model sentimen BERT?

Melalui penelitian ini, diharapkan dapat diperoleh solusi yang lebih efisien dan akurat dalam menangani kedua tugas secara simultan dengan memanfaatkan pendekatan pembelajaran mendalam berbasis *Fully Neural Network*.

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi model berbasis *Fully Neural Network* yang mampu melakukan klasifikasi sentimen dan pemodelan topik secara simultan menggunakan pendekatan *Multi-Task Learning* (MTL). Dengan pendekatan ini, model diharapkan dapat memanfaatkan hubungan antara kedua tugas tersebut untuk meningkatkan performa keseluruhan dan efisiensi dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP).

Secara spesifik, penelitian ini memiliki beberapa tujuan utama sebagai berikut:

1. Mengembangkan arsitektur berbasis *Fully Neural Network* yang dapat secara bersamaan melakukan klasifikasi sentimen dan pemodelan topik dalam satu model terpadu. Dengan integrasi ini, model diharapkan mampu menangkap hubungan semantik yang lebih dalam dan menghasilkan representasi teks yang lebih bermakna.
2. Menganalisis pengaruh *Multi-Task Learning* (MTL) terhadap efisiensi dan performa model dalam menangani kedua tugas secara simultan. Evaluasi akan dilakukan untuk memahami bagaimana strategi berbagi parameter antara dua tugas ini dapat meningkatkan akurasi, stabilitas, dan efisiensi model dibandingkan dengan pendekatan *single-task*.
3. Menerapkan *Autoencoder* untuk reduksi dimensi guna meningkatkan representasi fitur dan mengoptimalkan performa model *Fully Neural Network*. *Autoencoder* akan digunakan untuk mengekstraksi informasi laten dari teks secara lebih efektif, mengurangi redundansi fitur, dan membantu model dalam menangkap struktur data yang lebih informatif.

Melalui penelitian ini, diharapkan dapat diperoleh model yang lebih efisien, akurat, dan tangguh dalam menangani klasifikasi sentimen dan pemodelan topik secara simultan. Implementasi *Fully Neural Network* dengan MTL dan *Autoencoder* diharapkan dapat memberikan kontribusi dalam peningkatan teknik pemrosesan bahasa alami, terutama dalam konteks analisis teks skala besar dan kompleks.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis, yang diuraikan sebagai berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan mampu memperluas wawasan dalam bidang *Natural Language Processing* (NLP), terutama dalam hal perancangan model yang menggabungkan analisis sentimen dan pemodelan topik dalam satu arsitektur secara simultan. Penggunaan *Deep Embedded Clustering* (DEC) sebagai komponen pemodelan topik berbasis *unsupervised learning* dalam kerangka *joint model* diharapkan dapat menambah khasanah penelitian mengenai integrasi dua tugas yang berbeda dalam satu sistem pembelajaran. Penelitian ini diharapkan dapat menjadi acuan bagi penelitian berikutnya dalam membangun sistem analisis teks yang lebih terintegrasi dan efisien.

2. Manfaat Praktis

Hasil penelitian ini diharapkan dapat digunakan sebagai alat bantu dalam memahami isi ulasan teks secara lebih menyeluruh, baik dari sisi sentimen maupun topik yang terkandung di dalamnya, dalam satu kali proses inferensi. Sistem yang dikembangkan berpotensi diterapkan pada berbagai domain, seperti pemantauan ulasan produk, evaluasi layanan pelanggan, hingga analisis tren opini publik di platform digital. Pendekatan *joint model* ini juga diharapkan dapat mengurangi beban komputasi dibandingkan menjalankan kedua analisis secara terpisah.

1.5 Ruang Lingkup Penelitian

Batasan dalam penelitian ini ditetapkan sebagai berikut:

1. Penelitian ini menggunakan analisis sentimen biner yang terdiri atas dua kelas, yaitu positif dan negatif.
2. Data yang digunakan merupakan *dataset* IMDB dan Yelp yang berupa kumpulan teks ulasan, yang diperoleh melalui platform Hugging Face, masing-masing pada *dataset* IMDB dan *dataset* Yelp Polarity.
3. Jumlah data ulasan yang digunakan dalam penelitian ini dibatasi sebanyak 30.000 ulasan untuk masing-masing *dataset*, baik IMDB maupun Yelp.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang menjadi dasar penelitian. Pembahasan dimulai dari konsep *analisis sentimen* beserta tingkatan dan pendekatannya, dilanjutkan dengan *pendeteksian topik* dan berbagai metodenya. Selanjutnya diuraikan dasar-dasar *neural network*, mulai dari arsitektur, fungsi aktivasi, *backpropagation*, hingga metrik evaluasi. Bab ini juga membahas *Bidirectional Encoder Representations from Transformers* (BERT) sebagai model representasi teks berbasis *transformer*, *autoencoder* sebagai komponen kompresi representasi, serta *multi-task learning* sebagai kerangka pelatihan model untuk beberapa tugas sekaligus.

2.1 Analisis Sentimen

Platform daring seperti media sosial, forum diskusi, dan situs ulasan produk memungkinkan pengguna mengekspresikan pendapat secara bebas. Hal ini menghasilkan data opini dalam jumlah besar yang terus bertambah, sehingga muncul kebutuhan akan metode komputasional yang mampu mengolahnya secara otomatis.

Analisis sentimen, yang juga dikenal sebagai *opinion mining*, merupakan bidang dalam pemrosesan bahasa alami yang berfokus pada identifikasi dan ekstraksi opini, sentimen, serta emosi yang terkandung dalam teks (B. Liu, 2012). Secara umum, analisis sentimen mengklasifikasikan opini ke dalam beberapa kategori: positif, negatif, atau netral. Klasifikasi ini dapat dilakukan pada berbagai tingkatan, mulai dari keseluruhan dokumen, per kalimat, hingga aspek tertentu dalam teks. Pada penelitian ini, analisis dilakukan pada tingkat dokumen, artinya setiap ulasan diklasifikasikan ke dalam satu label sentimen. Pendekatan ini sesuai karena dataset yang digunakan, yakni Yelp dan IMDB, masing-masing berisi ulasan dari satu pengguna tentang satu entitas.

Analisis sentimen umumnya dilakukan melalui dua pendekatan (Pang, L. Lee, dan Vaithyanathan, 2002). Pendekatan berbasis leksikon menggunakan kamus sentimen yang berisi kata-kata beserta nilai sentimennya untuk menentukan sentimen suatu teks. Pendekatan berbasis *machine learning* melatih model pada data berlabel agar dapat mempelajari pola bahasa yang berkorelasi dengan sentimen tertentu. Pendekatan ini lebih mampu menangani variasi bahasa, konteks, dan domain yang beragam, terutama pada data berskala besar. Penelitian ini menggunakan pen-

dekatan berbasis *machine learning* sebagai landasan utama.

2.2 Pendeteksian Topik

Topik merujuk pada tema utama yang terkandung dalam sebuah teks. Pendeteksian topik bertujuan menemukan tema-tema yang tersebar dalam kumpulan dokumen tanpa label yang ditentukan sebelumnya (Blei, 2012). Setiap topik direpresentasikan sebagai distribusi probabilitas atas sekumpulan kata, dan maknanya dapat ditelusuri dari kata-kata yang paling sering muncul di dalamnya. Metode ini termasuk ke dalam *unsupervised learning* karena tidak memerlukan anotasi data.

Salah satu metode pendeteksian topik yang umum digunakan adalah BERTopic (Grootendorst, 2022), yang menggabungkan representasi kontekstual dari model bahasa besar dengan *clustering* untuk menghasilkan topik yang koheren secara semantis.

Pada penelitian ini, pendeteksian topik dilakukan dengan pendekatan *clustering*. Dokumen-dokumen yang berdekatan maknanya dikelompokkan ke dalam satu klaster, dan setiap klaster diinterpretasikan sebagai satu topik. Pengelompokan dilakukan pada ruang laten hasil *autoencoder*, sebuah jaringan saraf yang mengompresi teks menjadi representasi yang lebih padat namun tetap mempertahankan informasi semantiknya.

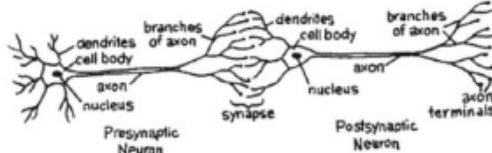
2.3 Neural Network

Jaringan saraf tiruan (*Artificial Neural Network/ANN*) adalah model komputasi yang dirancang meniru cara kerja sistem saraf biologis manusia (Aggarwal, 2018). Seperti neuron biologis yang saling terhubung melalui sinapsis, ANN terdiri dari unit-unit komputasi yang saling terhubung melalui bobot yang dapat disesuaikan. Proses pembelajaran berlangsung melalui penyesuaian bobot-bobot tersebut berdasarkan data pelatihan.

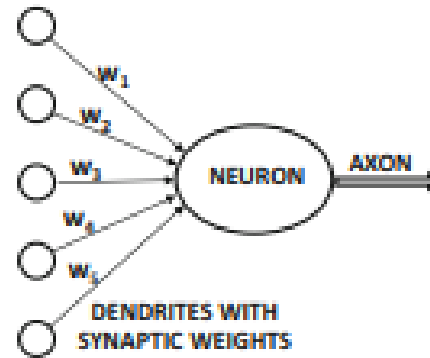
Secara umum, arsitektur ANN tersusun atas tiga komponen utama (Silva *et al.*, 2017). Pertama, *input layer*, yaitu lapisan yang menerima data mentah dari luar sistem dan meneruskannya ke lapisan berikutnya tanpa melakukan transformasi. Kedua, *hidden layer*, yaitu lapisan yang menerapkan fungsi aktivasi untuk mengidentifikasi pola kompleks dari data. Ketiga, *output layer*, yaitu lapisan yang menghasilkan prediksi akhir berdasarkan representasi yang telah dibangun oleh lapisan-lapisan sebelumnya.

Gambar 2.1 dan Gambar 2.2 menunjukkan perbandingan antara jaringan saraf

biologis dan ANN.



Gambar 2.1: Ilustrasi *neural network* secara biologis (Aggarwal, 2018)



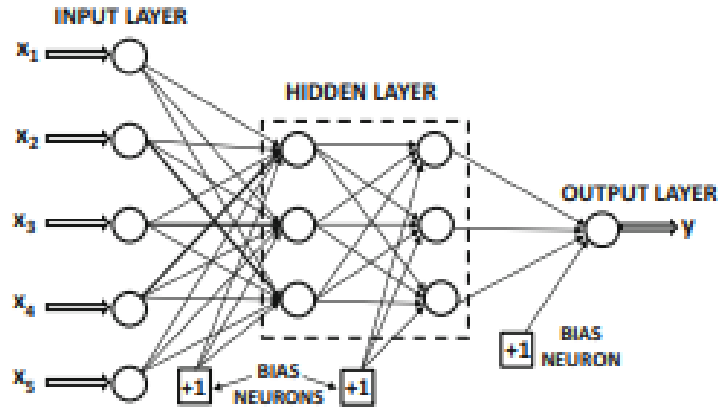
Gambar 2.2: *Artificial Neural Network* dengan representasi bobot sinapsis (Aggarwal, 2018)

Dalam bentuk paling sederhananya, sebuah *neural network* hanya memiliki satu lapisan komputasi yang disebut *perceptron*. Pada arsitektur ini, terdapat n neuron *input* yang meneruskan fitur ke satu neuron *output* melalui sekumpulan bobot. Nilai *output* diperoleh dari penjumlahan bobot yang dikalikan dengan masing-masing nilai *input*, ditambah sebuah nilai *bias* b yang berfungsi menggeser ambang batas keputusan model. Secara matematis, prediksi *output* dirumuskan sebagai:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b. \quad (2.1)$$

dengan $\mathbf{w}$ adalah vektor bobot, $\mathbf{x}$ adalah vektor input, dan b adalah skalar bias.

Perceptron hanya mampu memodelkan hubungan linier antara *input* dan *output*, sehingga tidak dapat menangani pola yang lebih kompleks. Untuk mengatasi keterbatasan ini, digunakan *multilayer neural network* yang memiliki satu atau lebih *hidden layer* di antara *input layer* dan *output layer*. Setiap lapisan menerima input dari lapisan sebelumnya dan meneruskannya searah menuju *output layer*. Pola aliran satu arah seperti ini dikenal sebagai *feed-forward*, dan arsitekturnya ditunjukkan pada Gambar 2.3.



Gambar 2.3: Arsitektur *feed-forward neural network* (Aggarwal, 2018)

Misalkan terdapat k *hidden layer* dengan masing-masing $p_1, \dots, p_k$ neuron, $\mathbf{x}$ sebagai vektor input berisi n neuron, $\mathbf{h}_1, \dots, \mathbf{h}_k$ sebagai keluaran tiap *hidden layer* setelah melewati fungsi aktivasi, dan $\mathbf{o}$ sebagai vektor output berisi m neuron. Matriks bobot $\mathbf{W}_1$ berdimensi $n \times p_1$ menghubungkan *input layer* dengan *hidden layer* pertama, $\mathbf{W}_r$ berdimensi $p_{r-1} \times p_r$ menghubungkan antar *hidden layer*, dan $\mathbf{W}_{k+1}$ berdimensi $p_k \times m$ menghubungkan *hidden layer* terakhir ke *output layer*. Proses komputasi pada tiap lapisan dirumuskan sebagai:

$$\mathbf{h}_1 = \phi_1(\mathbf{W}_1^\top \mathbf{x}), \quad (2.2)$$

$$\mathbf{h}_{r+1} = \phi_{r+1}(\mathbf{W}_{r+1}^\top \mathbf{h}_r), \quad r \in \{1, \dots, k-1\}, \quad (2.3)$$

$$\mathbf{o} = \phi_0(\mathbf{W}_{k+1}^\top \mathbf{h}_k). \quad (2.4)$$

dengan ϕ_i adalah fungsi aktivasi yang diterapkan pada lapisan ke- i . Output akhir $\mathbf{o}$ merupakan hasil transformasi bertingkat dari input $\mathbf{x}$ melalui seluruh lapisan jaringan secara berurutan.

2.3.1 Fungsi Aktivasi

Fungsi aktivasi digunakan dalam *neural network* untuk memberikan sifat non-linier pada model. Tanpa fungsi aktivasi, output jaringan hanya merupakan transformasi linier dari input sehingga tidak mampu menangkap pola yang kompleks (Goodfellow, Bengio, dan Courville, 2016). Berikut adalah fungsi aktivasi yang digunakan dalam penelitian ini.

2.3.1.1 Softmax

Fungsi *softmax* digunakan untuk mengubah vektor bernilai real menjadi distribusi probabilitas pada K kelas. Melalui proses ini, setiap nilai keluaran berada

pada rentang 0 hingga 1, dengan total seluruh probabilitas sama dengan 1. Untuk vektor masukan $\mathbf{v} = (v_1, v_2, \dots, v_K)$, fungsi *softmax* pada elemen ke- j didefinisikan sebagai:

$$\phi(\mathbf{v})_j = \frac{e^{v_j}}{\sum_{k=1}^K e^{v_k}}. \quad (2.5)$$

dengan e adalah bilangan Euler bernilai sekitar 2,718. Elemen dengan nilai v_j yang lebih besar menghasilkan e^{v_j} yang lebih besar, sehingga probabilitas $\phi(\mathbf{v})_j$ yang diperoleh juga lebih tinggi. Fungsi *softmax* umum digunakan pada lapisan keluaran jaringan untuk tugas klasifikasi multikelas karena keluarannya dapat langsung diinterpretasikan sebagai probabilitas prediksi tiap kelas.

2.3.1.2 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) mengatasi masalah *vanishing gradient* dengan cara meneruskan nilai *input* apa adanya jika positif, dan mengembalikan nol jika negatif:

$$\phi(v) = \max\{v, 0\}. \quad (2.6)$$

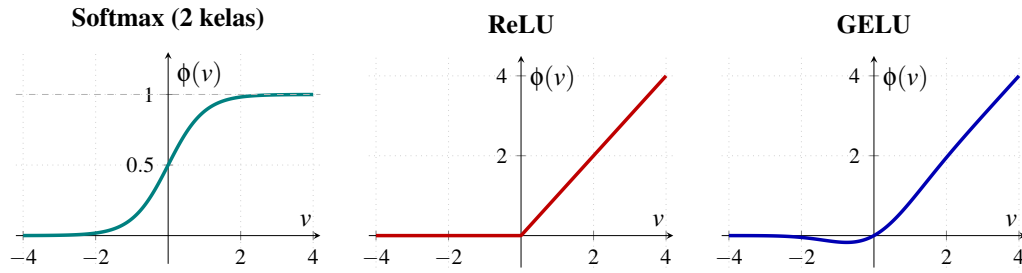
ReLU lebih efisien secara komputasi dan lebih mudah dilatih pada jaringan yang dalam, karena gradiennya selalu bernilai 1 untuk *input* positif sehingga informasi pembaruan tidak melemah saat merambat mundur melalui banyak lapisan.

2.3.1.3 Gaussian Error Linear Unit (GELU)

Gaussian Error Linear Unit (GELU) digunakan sebagai fungsi aktivasi pada arsitektur *Transformer* seperti BERT (Hendrycks dan Gimpel, 2016). Berbeda dengan ReLU yang mengubah seluruh nilai negatif menjadi nol, GELU mengatur kontribusi setiap nilai *input* berdasarkan probabilitas kumulatif distribusi Gaussian pada nilai tersebut:

$$\phi(v) = v \cdot \Phi(v) = v \cdot \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{v}{\sqrt{2}}\right) \right]. \quad (2.7)$$

dengan $\Phi(v)$ adalah fungsi distribusi kumulatif Gaussian standar, dan $\operatorname{erf}(\cdot)$ adalah fungsi error dengan rentang nilai $[-1, 1]$, sehingga $\Phi(v) \in [0, 1]$. Untuk v besar dan positif, GELU mendekati perilaku ReLU, sedangkan di sekitar nol transisinya berlangsung mulus.



Gambar 2.4: Grafik fungsi aktivasi: *Softmax*, ReLU, dan GELU

Perbandingan ketiga fungsi aktivasi ditunjukkan pada Gambar 2.4.

2.3.2 Fungsi Loss

Fungsi *loss* mengukur seberapa jauh prediksi model menyimpang dari nilai yang sebenarnya. Untuk tugas klasifikasi multikelas, *categorical cross-entropy loss* merupakan pilihan yang umum digunakan (Aggarwal, 2018):

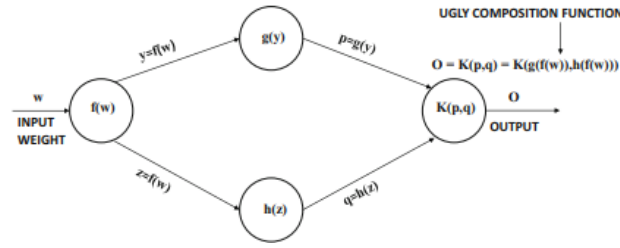
$$L = - \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij}). \quad (2.8)$$

dengan n adalah jumlah sampel, k adalah jumlah kelas, y_{ij} adalah label aktual dalam bentuk *one-hot encoded* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model untuk sampel ke- i pada kelas ke- j .

2.3.3 Backpropagation

Backpropagation adalah algoritma untuk menghitung gradien fungsi *loss* terhadap setiap bobot jaringan (Aggarwal, 2018), dengan merambatkan kesalahan dari *output* ke lapisan-lapisan sebelumnya menggunakan aturan rantai. Nilai gradien menunjukkan seberapa besar dan ke arah mana bobot perlu diubah agar nilai *loss* berkurang.

Aturan rantai memungkinkan penghitungan dampak perubahan kecil pada suatu bobot terhadap nilai *loss* akhir, meskipun terdapat banyak lapisan komputasi di antaranya. Misalkan terdapat jaringan sederhana seperti pada Gambar 2.5, di mana bobot w memengaruhi *output* o melalui dua jalur. Gradien fungsi *loss* terhadap w diperoleh dengan menjumlahkan kontribusi dari seluruh jalur:



Gambar 2.5: Ilustrasi graf komputasi *multi-layer perceptron* (Aggarwal, 2018)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \left(\frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial y} \cdot \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial w} \right), \quad (2.9)$$

dengan p dan q adalah skalar nilai antara dari dua jalur menuju o , dan y, z adalah skalar nilai pra-aktivasi pada masing-masing jalur. Setiap suku dalam tanda kurung merepresentasikan satu jalur dari w ke o .

Pola penjumlahan atas semua jalur berlaku umum. Untuk jaringan dengan *hidden unit* $\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_k$ dan bobot w_r yang menghubungkan lapisan ke- $(r-1)$ ke lapisan ke- r , gradien *loss* terhadap w_r dirumuskan sebagai:

$$\frac{\partial L}{\partial w_r} = \frac{\partial L}{\partial o} \left[\sum_{[\mathbf{h}_r, \dots, \mathbf{h}_k, o] \in \mathcal{P}} \frac{\partial o}{\partial \mathbf{h}_k} \prod_{i=r}^{k-1} \frac{\partial \mathbf{h}_{i+1}}{\partial \mathbf{h}_i} \right] \frac{\partial \mathbf{h}_r}{\partial w_r}, \quad r \in \{1, \dots, k\}, \quad (2.10)$$

dengan $\mathcal{P}$ adalah himpunan semua jalur yang mungkin dari $\mathbf{h}_r$ menuju o . Menghitung ekspresi dalam kurung siku secara langsung untuk setiap bobot sangat tidak efisien karena jumlah jalur dapat tumbuh eksponensial seiring bertambahnya lapisan. Untuk itu, ekspresi tersebut didefinisikan sebagai sinyal kesalahan:

$$\delta_r = \frac{\partial L}{\partial \mathbf{h}_r}, \quad (2.11)$$

sehingga gradien terhadap w_r dapat ditulis lebih ringkas sebagai:

$$\frac{\partial L}{\partial w_r} = \delta_r \cdot f'(z_r) \cdot \mathbf{h}_{r-1}, \quad (2.12)$$

dengan $f'(z_r)$ adalah skalar turunan fungsi aktivasi pada nilai pra-aktivasi $z_r = \mathbf{w}_r^\top \mathbf{h}_{r-1}$, dan $\mathbf{h}_{r-1}$ adalah vektor keluaran lapisan sebelumnya. Untuk memperbarui setiap bobot, yang diperlukan hanyalah nilai δ_r pada lapisan yang bersangkutan.

Nilai δ_r tidak dihitung dari nol untuk setiap lapisan, melainkan diwariskan secara rekursif dari lapisan yang lebih dekat ke *output*. Rekursi dimulai dari lapisan *output*, di mana $\delta_o = \frac{\partial L}{\partial o}$ sudah diketahui langsung dari fungsi *loss*. Dari sana, δ_r pada setiap lapisan tersembunyi dihitung dengan menjumlahkan kontribusi dari seluruh neuron pada lapisan di atasnya:

$$\delta_r = \sum_{h_{r+1} \in \mathbf{h}_{r+1}} f'(z_{r+1}) \mathbf{w}_{r+1} \delta_{r+1}. \quad (2.13)$$

Proses penyebaran mundur inilah yang menjadi asal nama *backpropagation*. Gradien yang diperoleh dari persamaan (2.12) kemudian digunakan untuk memperbarui parameter melalui teknik optimasi.

2.3.4 Teknik Optimasi

Proses optimasi merupakan tahapan pencarian parameter yang menghasilkan nilai fungsi *loss* sekecil mungkin selama pelatihan (A. Zhang *et al.*, 2023). Gradien yang diperoleh dari *backpropagation* digunakan untuk memperbarui bobot model secara iteratif.

2.3.4.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) adalah algoritma optimasi yang diperkenalkan oleh (Rumelhart, Hinton, dan Williams, 1986) untuk meminimalkan fungsi *loss* $L(\mathbf{w})$ secara iteratif dengan memperbarui bobot ke arah negatif gradiennya:

$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \eta \nabla L(\mathbf{w})^{[t]}. \quad (2.14)$$

dengan $\mathbf{w}^{[t]}$ adalah vektor bobot pada iterasi ke- t , $\nabla L(\mathbf{w})^{[t]}$ adalah vektor gradien fungsi *loss* terhadap bobot pada iterasi ke- t , dan η adalah skalar *learning rate* yang mengontrol besar langkah pembaruan. Nilai η yang terlalu kecil memperlambat konvergensi, sedangkan nilai yang terlalu besar dapat membuat pelatihan tidak stabil.

2.3.4.2 Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation (Adam), yang dikemukakan oleh (Kingma dan Ba, 2015), mengembangkan SGD dengan mengadaptasi *learning rate* secara individual untuk setiap parameter. Adaptasi ini didasarkan pada dua estimasi momen gradien: momen pertama $\mathbf{v}^{[t]}$ terhadap gradien itu sendiri, dan momen kedua $\mathbf{s}^{[t]}$ terhadap kuadrat gradien:

$$\mathbf{v}^{[t]} = \beta_1 \mathbf{v}^{[t-1]} + (1 - \beta_1) \nabla L(\mathbf{w})^{[t]}. \quad (2.15)$$

$$\mathbf{s}^{[t]} = \beta_2 \mathbf{s}^{[t-1]} + (1 - \beta_2) \left(\nabla L(\mathbf{w})^{[t]} \right)^2. \quad (2.16)$$

dengan β_1 dan β_2 adalah skalar koefisien peluruhan yang mengontrol pengaruh gradien lama terhadap estimasi saat ini. Karena kedua momen diinisialisasi dengan

nol, estimasinya cenderung bias ke nol pada iterasi awal. Koreksi bias dilakukan dengan normalisasi:

$$\hat{\mathbf{v}}^{[t]} = \frac{\mathbf{v}^{[t]}}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}^{[t]} = \frac{\mathbf{s}^{[t]}}{1 - \beta_2^t}. \quad (2.17)$$

Seiring bertambahnya t , nilai β_1^t dan β_2^t mendekati nol sehingga koreksi ini semakin tidak berpengaruh. Pembaruan bobot dilakukan menggunakan kedua momen yang telah dikoreksi:

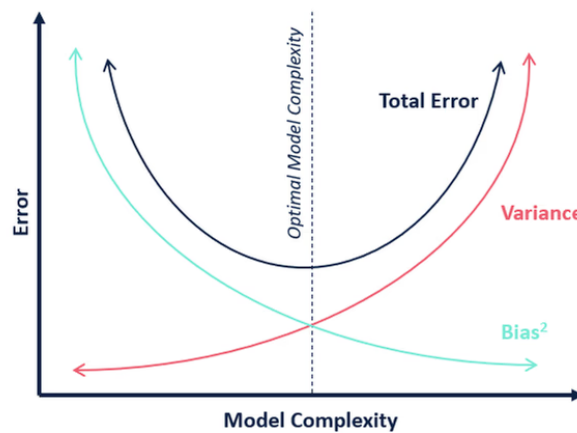
$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}^{[t]} + \epsilon}} \hat{\mathbf{v}}^{[t]}. \quad (2.18)$$

dengan $\epsilon = 10^{-8}$ adalah skalar untuk mencegah pembagian dengan nol (A. Zhang *et al.*, 2023). Pembagian dengan $\sqrt{\hat{\mathbf{s}}^{[t]}}$ memperkecil langkah pembaruan untuk parameter dengan gradien besar dan berfluktuasi, serta memperbesar untuk parameter dengan gradien kecil dan konsisten.

2.3.5 Regularisasi

Dalam melatih model, terdapat dua kondisi yang perlu dihindari, yaitu *underfitting* dan *overfitting*. *Underfitting* terjadi ketika model gagal mempelajari pola data pelatihan, ditandai dengan bias yang tinggi. *Overfitting* terjadi ketika model terlalu menghafalkan data pelatihan sehingga performanya menurun pada data baru, ditandai dengan variansi yang tinggi (Goodfellow, Bengio, dan Courville, 2016).

Gambar 2.6 menunjukkan hubungan antara kompleksitas model dengan bias, variansi, dan total error. Bias menurun seiring meningkatnya kompleksitas model, sementara variansi justru meningkat. Total error yang merupakan kombinasi keduanya mencapai nilai minimum pada titik kompleksitas optimal.



Gambar 2.6: Ilustrasi *bias-variance tradeoff* terhadap kompleksitas model (Aggarwal, 2018, telah diolah kembali)

Untuk mencegah *overfitting*, salah satu teknik yang digunakan adalah *weight decay* atau *L₂ regularization*. Teknik ini menambahkan penalti pada fungsi *loss* yang proporsional terhadap besar bobot model, sehingga model cenderung menghasilkan bobot bernilai kecil:

$$\tilde{L}(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2. \quad (2.19)$$

dengan $L(\mathbf{w})$ adalah fungsi *loss* asal, $\|\mathbf{w}\|^2$ adalah jumlah kuadrat seluruh bobot dalam model, dan λ adalah skalar yang mengontrol kekuatan penalti.

2.3.6 Metrik Evaluasi

Setelah model dilatih, performa klasifikasinya diukur menggunakan data yang belum pernah dilihat selama pelatihan. Dasar dari evaluasi ini adalah *confusion matrix*, yaitu tabel yang membandingkan prediksi model terhadap label aktual. Dari tabel ini diperoleh empat komponen dasar: *True Positive* (TP) yaitu sampel positif yang diprediksi benar, *True Negative* (TN) yaitu sampel negatif yang diprediksi benar, *False Positive* (FP) yaitu sampel negatif yang salah diprediksi sebagai positif, dan *False Negative* (FN) yaitu sampel positif yang salah diprediksi sebagai negatif.

Dari keempat komponen tersebut, diturunkan empat metrik utama. Akurasi mengukur proporsi keseluruhan prediksi yang benar:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.20)$$

Precision mengukur seberapa tepat model ketika menyatakan sesuatu sebagai positif, artinya dari semua yang diprediksi positif, berapa banyak yang benar-benar positif:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (2.21)$$

Recall mengukur seberapa lengkap model dalam menangkap semua sampel positif yang ada, artinya dari semua yang sebenarnya positif, berapa banyak yang berhasil ditemukan:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (2.22)$$

Kedua metrik ini sering kali saling bertentangan. Model dengan *precision* tinggi cenderung memiliki *recall* rendah, dan sebaliknya. *F1-Score* menggabungkan keduanya melalui rata-rata harmonik:

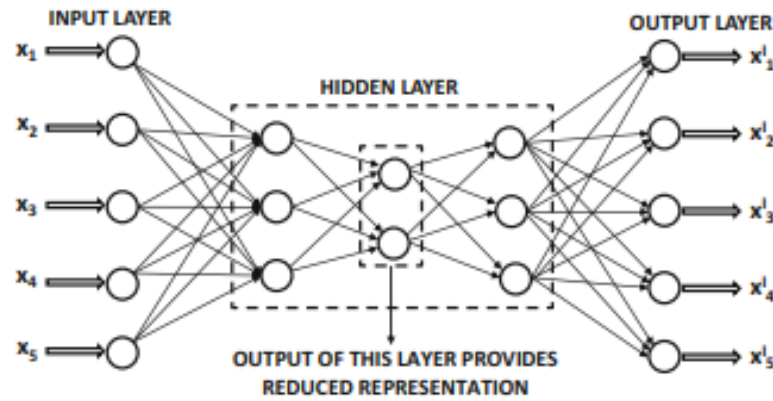
$$F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}. \quad (2.23)$$

F1-Score berguna pada dataset yang tidak seimbang karena tidak memberikan bobot berlebihan pada kelas yang mendominasi.

2.4 Autoencoder

Pemahaman tentang arsitektur *feed-forward neural network* menjadi landasan untuk memahami *autoencoder*, yang pada dasarnya adalah sebuah jaringan saraf yang dilatih untuk merekonstruksi inputnya (Goodfellow, Bengio, dan Courville, 2016). Karena tidak memerlukan label, *autoencoder* termasuk ke dalam kategori *unsupervised learning* (Hinton dan Salakhutdinov, 2006).

Arsitektur *autoencoder* (Vincent *et al.*, 2010) terdiri dari dua komponen utama: *encoder* yang memetakan representasi input dari dimensi tinggi ke dimensi yang lebih rendah, dan *decoder* yang merekonstruksi representasi tersebut ke dimensi semula. Lapisan tersempit di antara keduanya disebut *bottleneck* atau ruang laten. Arsitektur lengkapnya ditunjukkan pada Gambar 2.7.



Gambar 2.7: Arsitektur *autoencoder* menunjukkan proses *encoding* dan *decoding* (Aggarwal, 2018)

2.4.1 Encoder dan Decoder

Secara formal, misalkan $\mathbf{x} \in \mathbb{R}^D$ adalah vektor input berdimensi D . *Encoder* memetakan $\mathbf{x}$ ke dalam vektor laten $\mathbf{z} \in \mathbb{R}^d$ dengan $d < D$:

$$\mathbf{z} = f_{\theta}(\mathbf{x}) = \sigma_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h). \quad (2.24)$$

dengan $\mathbf{W}_h \in \mathbb{R}^{d \times D}$ adalah matriks bobot *encoder*, $\mathbf{b}_h \in \mathbb{R}^d$ adalah vektor bias, dan

$\sigma_h(\cdot)$ adalah fungsi aktivasi. Simbol θ merepresentasikan seluruh parameter pada bagian *encoder*.

Selanjutnya, *decoder* memetakan kembali vektor laten $\mathbf{z}$ ke dalam ruang dimensi asal untuk menghasilkan rekonstruksi $\hat{\mathbf{x}} \in \mathbb{R}^D$:

$$\hat{\mathbf{x}} = g_{\theta'}(\mathbf{z}) = \sigma_o(\mathbf{W}_o \mathbf{z} + \mathbf{b}_o). \quad (2.25)$$

dengan $\mathbf{W}_o \in \mathbb{R}^{D \times d}$ adalah matriks bobot *decoder*, $\mathbf{b}_o \in \mathbb{R}^D$ adalah vektor bias, dan $\sigma_o(\cdot)$ adalah fungsi aktivasi output. Simbol θ' merepresentasikan seluruh parameter pada bagian *decoder*. Secara keseluruhan, *autoencoder* melakukan transformasi:

$$\hat{\mathbf{x}} = g_{\theta'}(f_{\theta}(\mathbf{x})). \quad (2.26)$$

2.4.2 Fungsi Objektif

Autoencoder dilatih dengan meminimalkan selisih antara *input* asal dan hasil rekonstruksinya. Pada penelitian ini, fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE):

$$\mathcal{L}(\theta, \theta') = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - g_{\theta'}(f_{\theta}(\mathbf{x}_i))\|^2. \quad (2.27)$$

dengan n adalah jumlah sampel dan $\|\cdot\|^2$ adalah kuadrat norma Euclidean. Parameter *encoder* θ dan *decoder* θ' diperbarui secara bersama-sama menggunakan *backpropagation* dan optimasi Adam (Vincent *et al.*, 2010).

2.5 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) merupakan model bahasa yang diperkenalkan oleh (Devlin *et al.*, 2018). Berbeda dari model bahasa sebelumnya yang membaca teks secara satu arah, BERT memahami makna suatu kata dengan mempertimbangkan konteks dari kiri maupun kanan secara bersamaan. Kemampuan ini dimungkinkan oleh arsitekturnya yang didasarkan pada *Transformer Encoder*, yaitu mekanisme yang memungkinkan setiap token menghitung relevansinya terhadap token lain dalam kalimat secara paralel.

Sebelum digunakan, BERT dilatih pada korpus teks berskala besar, yaitu Wikipedia bahasa Inggris dan BookCorpus. Pada penelitian ini, bobot BERT yang sudah terlatih tidak diperbarui selama pelatihan. BERT digunakan murni seba-

gai ekstraktor fitur yang dibekukan (*frozen*), sehingga representasi yang dihasilkan langsung digunakan sebagai fitur untuk tahap berikutnya.

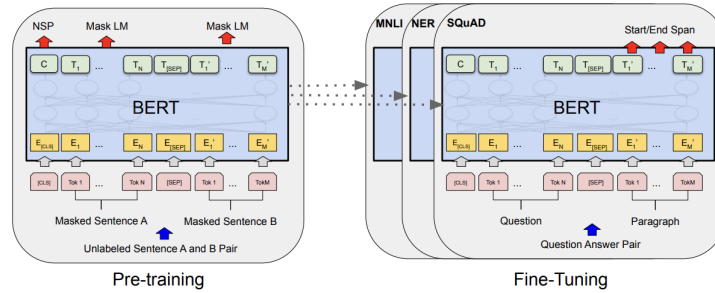
2.5.1 Arsitektur BERT

Arsitektur BERT tersusun dari L lapisan *Transformer Encoder* yang identik, masing-masing terdiri dari dua sub-lapisan: *Multi-Head Self-Attention* dan *Position-wise Feed-Forward Network*. Keluaran setiap lapisan, seperti ditunjukkan pada Gambar 2.8, dihitung sebagai:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell-1)} + \text{MultiHeadAttention}(\mathbf{H}^{(\ell-1)})\right), \quad (2.28)$$

$$\mathbf{H}_{\text{out}}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell)} + \text{FFN}(\mathbf{H}^{(\ell)})\right). \quad (2.29)$$

dengan $\mathbf{H}^{(\ell)}$ adalah matriks *hidden state* pada lapisan ke- ℓ , dan LayerNorm adalah normalisasi lapisan yang membantu stabilitas pelatihan. Jumlah lapisan L , dimensi *hidden state* H , dan jumlah *attention head* A bergantung pada varian model yang dipilih.



Gambar 2.8: Arsitektur BERT dengan beberapa lapisan *Transformer Encoder* (Devlin *et al.*, 2018)

2.5.2 Mekanisme *Self-Attention*

BERT menggunakan mekanisme *self-attention* untuk menentukan seberapa penting suatu kata terhadap kata-kata lain dalam kalimat yang sama. Input $\mathbf{X} \in \mathbb{R}^{n \times d}$ dengan n token dan dimensi d diproyeksikan ke tiga representasi melalui matriks bobot yang dipelajari saat pelatihan, yaitu *query* $\mathbf{Q}$, *key* $\mathbf{K}$, dan *value* $\mathbf{V}$:

$$\mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{n \times d_k}, \quad (2.30)$$

$$\mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{n \times d_k}, \quad (2.31)$$

$$\mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{n \times d_v}. \quad (2.32)$$

dengan $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ dan $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$ adalah matriks bobot yang dapat dipelajari. *Query* merepresentasikan kata yang sedang mencari konteks, *key* merepresentasikan kata-kata yang berpotensi menjadi konteks, dan *value* adalah informasi yang diambil apabila konteks tersebut dianggap relevan.

Skor relevansi antara setiap pasangan kata dihitung melalui perkalian *query* dan *key*, lalu dinormalisasi dengan $\sqrt{d_k}$ untuk menjaga skala nilai. Hasilnya kemudian diubah menjadi distribusi probabilitas menggunakan *softmax*:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.33)$$

Untuk meningkatkan kemampuan representasi, BERT menjalankan mekanisme *attention* ini secara paralel sebanyak h kali pada *subspace* yang berbeda, yang disebut *Multi-Head Attention*. Setiap hasil digabungkan dan diproyeksikan kembali ke dimensi semula:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O. \quad (2.34)$$

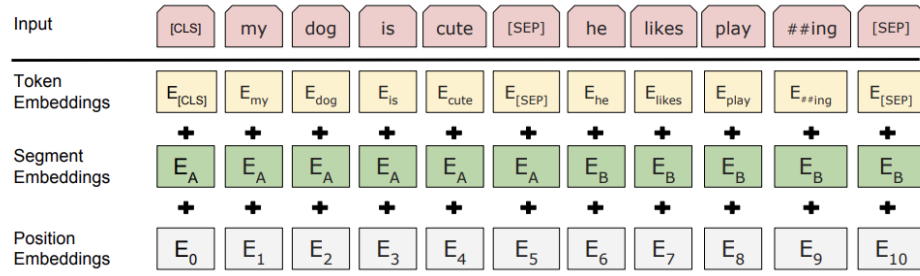
dengan $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q^i, \mathbf{K}\mathbf{W}_K^i, \mathbf{V}\mathbf{W}_V^i)$. Pada BERT-Base, $h = 12$ dan $d_k = d_v = 64$.

2.5.3 Representasi Input BERT

Sebelum teks dapat diproses oleh BERT, setiap token perlu diubah menjadi vektor numerik. BERT merepresentasikan setiap token sebagai penjumlahan dari tiga jenis *embedding* yang masing-masing membawa informasi berbeda:

$$\mathbf{E}_i = \mathbf{E}_{\text{token}}^{(i)} + \mathbf{E}_{\text{segment}}^{(i)} + \mathbf{E}_{\text{position}}^{(i)}. \quad (2.35)$$

Token embedding merepresentasikan makna semantik dari setiap kata atau *sub-word*. BERT menggunakan tokenisasi WordPiece dengan kosakata sebesar 30.000 token, sehingga kata yang tidak dikenal dapat diurai menjadi unit yang lebih kecil. *Segment embedding* menandai asal kalimat dari setiap token, yang berguna ketika *input* terdiri dari dua kalimat. *Position embedding* merepresentasikan posisi setiap token dalam urutan, sebab mekanisme *self-attention* pada dasarnya tidak mengandung informasi urutan. Berbeda dari Transformer asli yang menggunakan rumus sinusoidal, BERT mempelajari *position embedding* langsung dari data dengan panjang urutan maksimum token. Ilustrasi representasi input BERT ditunjukkan pada Gambar 2.9.



Gambar 2.9: Representasi *input* BERT yang terdiri dari *token*, *segment*, dan *position embedding* (Devlin *et al.*, 2018)

Selain token biasa, BERT menyisipkan token khusus [CLS] di awal setiap urutan *input*. Token ini dirancang untuk mengakumulasi informasi dari seluruh token dalam kalimat melalui lapisan-lapisan *self-attention*. Pada penelitian ini, representasi token [CLS] pada lapisan terakhir BERT inilah yang digunakan sebagai vektor fitur untuk setiap ulasan.

Pada penelitian ini, BERT digunakan sepenuhnya sebagai ekstraktor fitur dengan bobot yang dibekukan. Setiap ulasan teks dimasukkan ke dalam BERT dengan panjang maksimum token, kemudian representasi *hidden state* pada posisi token [CLS] dari lapisan terakhir diambil sebagai fitur:

$$\mathbf{f} = \mathbf{H}_{[CLS]}^{(L)} \in \mathbb{R}^d. \quad (2.36)$$

dengan $\mathbf{H}_{[CLS]}^{(L)}$ adalah vektor *hidden state* token [CLS] pada lapisan ke- L , dan d adalah dimensi *hidden state* yang bergantung pada varian model yang digunakan. Vektor $\mathbf{f}$ ini selanjutnya menjadi representasi numerik dari setiap ulasan dan digunakan sebagai masukan *autoencoder* pada tahap berikutnya.

2.6 Multi-Task Learning

Multi-Task Learning (MTL) merupakan pendekatan dalam *machine learning* yang bertujuan meningkatkan kemampuan generalisasi model dengan memanfaatkan informasi dari beberapa tugas yang saling berkaitan secara bersamaan (Caruana, 1997). Ketika dua tugas memiliki keterkaitan, informasi pembelajaran dari satu tugas dapat membantu tugas lainnya karena keduanya berbagi pola atau struktur yang sama dalam data.

Konsep kunci yang mendasari MTL adalah *inductive transfer*, yaitu kemampuan model untuk memindahkan pengetahuan yang dipelajari dari satu tugas ke tugas lainnya. (Caruana, 1997) menjelaskan bahwa MTL mencapai hal ini dengan cara mempelajari beberapa tugas secara paralel menggunakan representasi

bersama. Tugas-tugas lain berfungsi sebagai *inductive bias*, yaitu mekanisme yang mengarahkan model agar mempelajari hipotesis yang dapat menjelaskan lebih dari satu tugas sekaligus (Ruder, 2017).

Pada penelitian ini, MTL digunakan untuk mempelajari analisis sentimen dan pendeteksian topik secara bersamaan. Kedua tugas ini memiliki keterkaitan yang kuat karena sama-sama bergantung pada pemahaman semantis teks ulasan, sehingga representasi yang berguna untuk mendeteksi topik juga cenderung berguna untuk mengklasifikasikan sentimen.

2.6.1 Formulasi MTL

Misalkan terdapat T tugas yang dipelajari secara bersamaan. Untuk setiap tugas $t \in \{1, 2, \dots, T\}$, terdapat dataset $\mathcal{D}_t = \{(\mathbf{x}_i^{(t)}, y_i^{(t)})\}_{i=1}^{n_t}$ dan fungsi *loss* $\mathcal{L}_t$ yang mengukur kesalahan prediksi pada tugas ke- t . Tujuan MTL adalah meminimalkan kombinasi bobot dari *loss* semua tugas secara bersamaan:

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \alpha_t \mathcal{L}_t(\theta_t, \theta_{\text{shared}}). \quad (2.37)$$

dengan α_t adalah skalar bobot yang mengontrol seberapa besar kontribusi tugas ke- t terhadap total *loss*, θ_t adalah parameter yang spesifik untuk tugas ke- t , dan θ_{shared} adalah parameter yang dibagikan bersama oleh semua tugas. Parameter optimal diperoleh dengan meminimalkan persamaan (2.37) secara bersama-sama:

$$\theta_{\text{shared}}^*, \{\theta_t^*\}_{t=1}^T = \arg \min_{\theta_{\text{shared}}, \{\theta_t\}} \mathcal{L}_{\text{MTL}}. \quad (2.38)$$

Pada penelitian ini, fungsi *loss* total terdiri dari dua komponen. Komponen pertama adalah *clustering loss* berbasis *Kullback-Leibler Divergence* (KLD), yang mengoptimalkan jarak antara distribusi penugasan kluster Q dan distribusi target P . Komponen kedua adalah *sentiment loss* berbasis *categorical cross-entropy* untuk klasifikasi sentimen. Keduanya digabungkan sebagai berikut:

$$\mathcal{L}_{\text{MTL}} = \alpha_1 \cdot \mathcal{L}_{\text{cluster}} + \alpha_2 \cdot \mathcal{L}_{\text{sentiment}}. \quad (2.39)$$

dengan α_1 dan α_2 adalah skalar hiperparameter yang mengontrol bobot masing-masing komponen. *Clustering loss* diformulasikan sebagai KLD antara distribusi target P dan distribusi penugasan Q :

$$\mathcal{L}_{\text{cluster}} = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}. \quad (2.40)$$

dengan q_{ij} adalah probabilitas sampel ke- i termasuk ke kluster ke- j , dan p_{ij} adalah distribusi target yang diturunkan dari Q sebagai berikut:

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} q_{ij'}^2 / \sum_i q_{ij'}}. \quad (2.41)$$

Pembagian dengan $\sum_i q_{ij}$ menormalisasi frekuensi kluster agar kluster yang besar tidak mendominasi, sedangkan operasi kuadrat pada q_{ij} membuat distribusi keanggotaan lebih terkonsentrasi pada satu kluster.

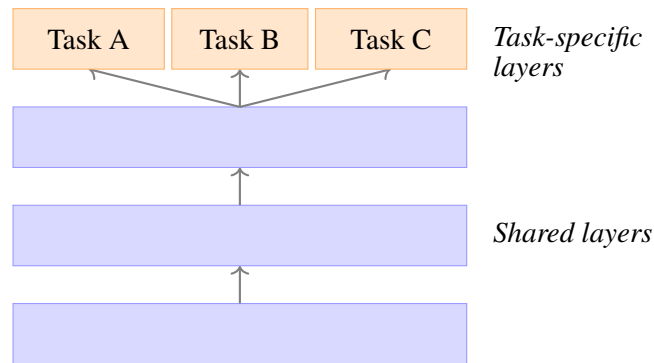
2.6.2 Hard Parameter Sharing

Hard parameter sharing merupakan pendekatan MTL yang paling umum digunakan dalam *deep learning* (Ruder, 2017). Pada pendekatan ini, lapisan tersembunyi bersifat *shared* antar semua tugas, sementara setiap tugas memiliki lapisan *output* tersendiri. Secara formal, untuk input $\mathbf{x}$, proses komputasinya adalah:

$$\mathbf{h}_{\text{shared}} = f_{\text{shared}}(\mathbf{x}; \theta_{\text{shared}}), \quad (2.42)$$

$$\hat{y}_t = f_t(\mathbf{h}_{\text{shared}}; \theta_t), \quad t = 1, \dots, T. \quad (2.43)$$

dengan f_{shared} adalah jaringan bersama yang menghasilkan vektor representasi $\mathbf{h}_{\text{shared}}$, dan f_t adalah jaringan spesifik tugas ke- t . (Baxter, 2000) menunjukkan secara teoritis bahwa mekanisme ini mengurangi risiko *overfitting* karena gradien dari semua tugas mengalir ke lapisan bersama selama *backpropagation*. Arsitektur *hard parameter sharing* diilustrasikan pada Gambar 2.10.



Gambar 2.10: Ilustrasi arsitektur *hard parameter sharing* dalam *Multi-Task Learning* (Ruder, 2017, telah diolah kembali)

Pada penelitian ini, pendekatan *hard parameter sharing* diterapkan dengan menjadikan *autoencoder* sebagai jaringan bersama. Representasi laten $\mathbf{z}$ yang dihasilkan oleh *encoder* digunakan secara bersamaan oleh dua *head*: satu untuk pendeteksian

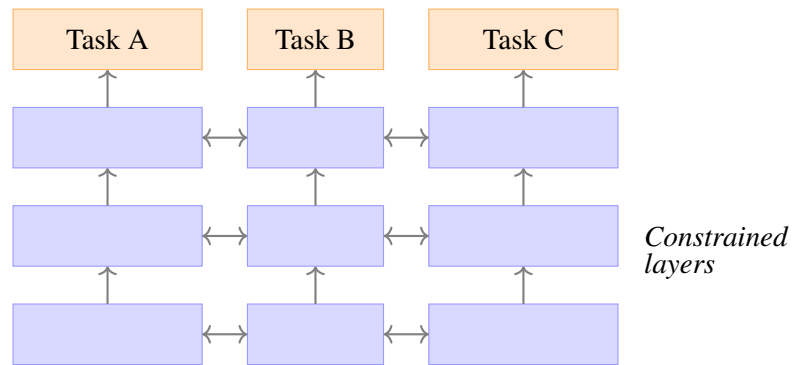
topik melalui *clustering layer*, dan satu lagi untuk klasifikasi sentimen melalui *sentiment classifier*.

2.6.3 Soft Parameter Sharing

Soft parameter sharing mengambil pendekatan yang berbeda. Setiap tugas memiliki model tersendiri dengan parameter masing-masing, namun parameter antar model diatur agar tidak terlalu jauh berbeda melalui regularisasi (Ruder, 2017). Salah satu cara yang umum digunakan adalah menambahkan penalti ℓ_2 terhadap perbedaan parameter antar tugas:

$$\mathcal{L}_{\text{soft}} = \sum_{t=1}^T \mathcal{L}_t(\theta_t) + \lambda \sum_{t=1}^T \sum_{t'=t+1}^T \|\theta_t - \theta_{t'}\|_2^2. \quad (2.44)$$

dengan λ adalah skalar koefisien regularisasi yang mengontrol seberapa mirip parameter antar tugas harus dijaga, dan $\|\theta_t - \theta_{t'}\|_2^2$ mengukur seberapa jauh kedua set parameter berbeda. Arsitektur *soft parameter sharing* diilustrasikan pada Gambar 2.11.



Gambar 2.11: Ilustrasi arsitektur *soft parameter sharing* (Ruder, 2017, telah diolah kembali)

Perbedaan mendasar antara keduanya terletak pada bagaimana pengetahuan dibagikan. Pada *hard parameter sharing*, bobot benar-benar identik sehingga semua tugas menggunakan representasi yang sama persis. Pada *soft parameter sharing*, setiap tugas tetap memiliki ruang belajar sendiri namun didorong untuk tidak terlalu menyimpang, sehingga lebih fleksibel namun membutuhkan lebih banyak parameter dan pengaturan λ yang tepat.

2.7 Clustering

Clustering merupakan teknik *unsupervised learning* yang mengelompokkan data berdasarkan kemiripan tanpa memerlukan label (Bezdek, 1984). Berdasarkan

cara pengelompokannya, terdapat dua pendekatan utama. *Hard clustering* membagi setiap titik data ke satu kluster, sedangkan *soft clustering* memungkinkan satu titik data masuk ke beberapa kluster sekaligus.

Metode *clustering* dapat diklasifikasikan ke dalam beberapa pendekatan, di antaranya *centroid-based*, *distribution-based*, *connectivity-based*, dan *density-based*. Penelitian ini menggunakan pendekatan *centroid-based clustering*, yaitu metode yang memisahkan data berdasarkan jarak setiap titik data terhadap pusat kluster (*centroid*) (Yusdiansyah dan Suryadi, 2019). Untuk mengukur jarak tersebut digunakan jarak Euclidean:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}. \quad (2.45)$$

dengan $\mathbf{p} = (p_1, \dots, p_D)$ dan $\mathbf{q} = (q_1, \dots, q_D)$ adalah dua vektor dalam ruang berdimensi D .

Penelitian ini menggunakan *Deep Embedded Clustering* (DEC) (Xie, Girshick, dan Farhadi, 2016) sebagai metode pendeteksian topik, yang mempelajari representasi laten dan melakukan *clustering* secara bersamaan. DEC menerapkan *soft clustering* melalui distribusi probabilitas penugasan kluster, sehingga setiap ulasan dapat terhubung ke lebih dari satu topik. Sebagai inisialisasi, DEC menggunakan K-Means untuk menentukan pusat kluster awal di ruang *embedding*.

2.7.1 K-Means Clustering

K-Means merupakan algoritma *centroid-based clustering* yang mengelompokkan data dengan meminimalkan total jarak antara setiap titik data dan pusat klasternya, dan termasuk ke dalam kategori *hard clustering* (MacQueen, 1967).

Secara formal, misalkan terdapat himpunan data $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_n \in \mathbb{R}^D$ dan jumlah kluster yang diinginkan adalah K . K-Means meminimalkan fungsi objektif berikut:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2. \quad (2.46)$$

dengan $\boldsymbol{\mu}_k$ adalah vektor *centroid* kluster ke- k , dan r_{nk} adalah skalar variabel indikator biner yang bernilai 1 jika titik data $\mathbf{x}_n$ termasuk kluster ke- k dan 0 jika tidak. Nilai J mengukur total kuadrat jarak seluruh titik data ke pusat klasternya masing-masing.

Proses optimasi dilakukan secara iteratif melalui dua tahap yang bergantian.

Pada tahap pertama (*assignment step*), setiap titik data dialokasikan ke kluster dengan *centroid* terdekat:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{yang lainnya.} \end{cases} \quad (2.47)$$

Pada tahap kedua (*update step*), *centroid* setiap kluster diperbarui menjadi rata-rata dari seluruh titik yang menjadi anggotanya:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}. \quad (2.48)$$

Pembilang adalah jumlah posisi seluruh anggota kluster ke- k , sedangkan penyebut adalah jumlah anggotanya. Dua tahap ini diulang sampai nilai J tidak berubah signifikan atau jumlah iterasi mencapai batas maksimum. Algoritma lengkapnya dapat dilihat pada Tabel 2.1.

Tabel 2.1: Algoritma K-Means *Clustering*

Algoritma K-Means <i>Clustering</i>	
INPUT: Himpunan data $\{\mathbf{x}_i\}_{i=1}^N$, banyak kluster K , batas toleransi ϵ , dan maksimal iterasi maxiter	
1. Inisialisasi secara acak himpunan <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$	
2. Tetapkan iter = 0	
3. REPEAT	
4. Tetapkan iter = iter + 1	
5. <i>Assignment step:</i> Cari nilai $r_{nk} = \begin{cases} 1, & k = \arg \min_j \ \mathbf{x}_n - \boldsymbol{\mu}_j\ ^2 \\ 0, & \text{yang lainnya} \end{cases}$	
6. <i>Update step:</i> Cari nilai $\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}$	
7. Hitung $J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \ \mathbf{x}_n - \boldsymbol{\mu}_k\ ^2$	
8. UNTIL $J < \epsilon$ OR iter > maxiter	
9. Untuk setiap $n = 1, 2, \dots, N$, tetapkan nilai $u_n = \arg \max_k (r_{nk})$, dengan $k = 1, 2, \dots, K$	
10. Bentuk $\{C_1, C_2, \dots, C_K\}$, dengan C_k merupakan himpunan anggota data $\mathbf{x}_i$ di mana $u_i = k$	
11. STOP	
OUTPUT: Nilai keanggotaan $\{u_n\}_{n=1}^N$ dan himpunan kluster $\{C_k\}_{k=1}^K$	

2.8 Interpretabilitas Model *Deep Learning*

Interpretabilitas model *deep learning* merujuk pada kemampuan untuk memahami dan menjelaskan keputusan yang dihasilkan oleh model yang bersifat *black-box* (Sundararajan, Taly, dan Yan, 2017). Dalam konteks pemrosesan bahasa alami,

interpretabilitas penting karena model perlu mampu menjelaskan mengapa suatu teks diklasifikasikan ke dalam kategori tertentu, terutama pada aplikasi analisis sentimen dan pengelompokan topik.

Secara umum, metode interpretabilitas dapat dikategorikan menjadi dua kelompok (J. Li *et al.*, 2016). Kelompok pertama adalah metode *intrinsic*, yaitu model yang secara bawaan dapat diinterpretasi seperti pohon keputusan dan regresi linier. Kelompok kedua adalah metode *post-hoc*, yaitu metode yang diterapkan setelah model dilatih untuk menganalisis perilaku model yang kompleks tanpa mengubah arsitekturnya.

Dalam penelitian ini, metode *post-hoc* digunakan untuk menganalisis kontribusi setiap token terhadap prediksi sentimen model, yaitu *occlusion-based token attribution* dan *Integrated Gradients*.

2.8.1 Occlusion-based Token Attribution

Occlusion-based attribution merupakan salah satu metode *post-hoc* yang banyak digunakan untuk menginterpretasi model berbasis teks (Zeiler dan Fergus, 2014; J. Li *et al.*, 2016). Metode ini mengukur pentingnya suatu token dengan cara menggantinya dan mengamati perubahan prediksi model.

Pada model berbasis *transformer* seperti BERT, oklusi dilakukan dengan mengganti token target menggunakan token khusus [MASK], sehingga model tetap menerima input dengan panjang yang sama. Pendekatan ini dikenal sebagai *leave-one-out attribution*.

Secara formal, diberikan teks input $\mathbf{x} = (x_1, x_2, \dots, x_T)$ yang terdiri dari T token, representasi kontekstual diperoleh melalui BERT sebagai:

$$\mathbf{h} = \text{BERT}(\mathbf{x}) \in \mathbb{R}^d. \quad (2.49)$$

dengan $\mathbf{h}$ merupakan vektor representasi [CLS] dari lapisan terakhir BERT. Representasi ini kemudian diteruskan ke lapisan klasifikasi sentimen untuk menghasilkan distribusi probabilitas:

$$f(\mathbf{x}) = \text{softmax}(\mathbf{W} \cdot \mathbf{h} + \mathbf{b}) \in \mathbb{R}^C. \quad (2.50)$$

dengan C adalah jumlah kelas sentimen, $\mathbf{W}$ adalah matriks bobot dan $\mathbf{b}$ adalah vektor bias lapisan klasifikasi. Skor atribusi token x_i terhadap kelas sentimen c didefinisikan sebagai selisih probabilitas prediksi sebelum dan sesudah oklusi:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}). \quad (2.51)$$

dengan $f_c(\mathbf{x})$ adalah skalar probabilitas prediksi kelas c pada teks asli, dan $f_c(\mathbf{x}_{\setminus i})$ adalah skalar probabilitas prediksi setelah token x_i diganti dengan [MASK]. Nilai positif berarti token x_i mendukung prediksi kelas c , nilai negatif berarti token menekannya, dan nilai mendekati nol berarti token tidak berpengaruh signifikan.

Dibandingkan dengan metode berbasis gradien, *occlusion-based attribution* lebih mudah diinterpretasi karena tidak bergantung pada komputasi turunan yang kompleks. Namun, metode ini memiliki kompleksitas waktu $O(T)$ terhadap panjang token sehingga lebih mahal secara komputasi untuk teks yang panjang (J. Li *et al.*, 2016).

2.8.2 Integrated Gradients

Integrated Gradients (IG) merupakan metode atribusi berbasis gradien yang diusulkan oleh (Sundararajan, Taly, dan Yan, 2017). Berbeda dengan *occlusion-based attribution* yang bersifat diskrit, IG menghitung kontribusi setiap dimensi input secara kontinu dengan mengintegrasikan gradien prediksi model sepanjang jalur lurus dari *baseline* menuju input asli.

Secara formal, diberikan vektor input $\mathbf{x} \in \mathbb{R}^d$ dan vektor *baseline* $\mathbf{x}' \in \mathbb{R}^d$ yang umumnya berupa vektor nol, atribusi IG untuk dimensi ke- i didefinisikan sebagai:

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha. \quad (2.52)$$

dengan α adalah skalar parameter yang menunjukkan posisi pada jalur dari *baseline* menuju input asli, dan $\frac{\partial f}{\partial x_i}$ adalah skalar gradien prediksi terhadap dimensi ke- i . Dalam praktiknya, integral tersebut diaproksimasi menggunakan *Riemann sum* dengan m langkah interpolasi:

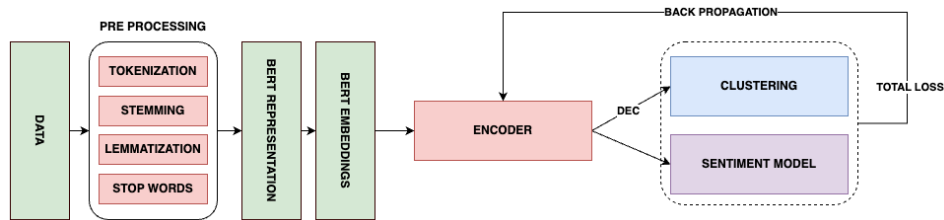
$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f\left(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}')\right)}{\partial x_i} \times \frac{1}{m}. \quad (2.53)$$

Metode IG memenuhi dua karakteristik penting, yaitu *sensitivity* dan *implementation invariance* (Sundararajan, Taly, dan Yan, 2017). *Sensitivity* menjamin bahwa apabila suatu fitur mempengaruhi prediksi model, fitur tersebut akan mendapatkan atribusi bernilai tidak nol. *Implementation invariance* menjamin bahwa dua model yang secara fungsional ekuivalen akan menghasilkan atribusi yang sama meskipun memiliki implementasi internal yang berbeda.

BAB 3

METODE MODEL JOINT SENTIMENT-TOPIC BERBASIS MULTITASK LEARNING

Bab ini menjelaskan rancangan metode yang digunakan dalam penelitian, mencakup pembentukan representasi teks menggunakan BERT, perancangan arsitektur JST-DEC berbasis multitask learning, proses pelatihan model, interpretasi hasil menggunakan metode atribusi, serta evaluasi kualitas topik yang dihasilkan.

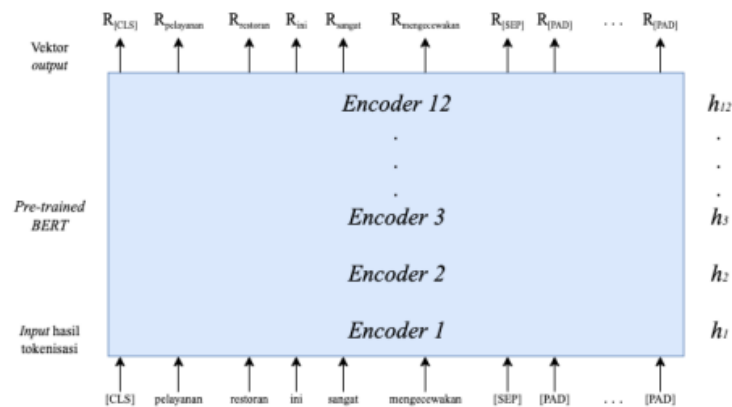


Gambar 3.1: Alur metode analisis sentimen dan pendeteksian topik

3.1 *Bidirectional Encoder Representations from Transformers* (BERT) Sebagai Model Representasi Teks

Terdapat dua pendekatan utama dalam pemanfaatan model BERT, yaitu *feature-based* dan *fine-tuning*. Pada pendekatan *feature-based*, BERT berfungsi sebagai *feature extractor* tanpa mengubah parameter internalnya. Pada pendekatan *fine-tuning*, lapisan keluaran khusus ditambahkan dan seluruh bobot dilatih ulang secara *end-to-end*.

Pada penelitian ini, BERT digunakan melalui pendekatan *feature-based* dengan bobot yang dibekukan sepenuhnya (*frozen*). Representasi yang dihasilkan kemudian digunakan sebagai *input* untuk *autoencoder* dan pengelompokan topik menggunakan DEC. Contoh representasi teks yang dihasilkan BERT ditunjukkan pada Gambar 3.2.



Gambar 3.2: Representasi teks menggunakan BERT ((Devlin *et al.*, 2018); telah diolah kembali).

Sebagai ilustrasi, misalkan kalimat *input* yang digunakan adalah “*pelayanan restoran ini sangat mengecewakan*”. Proses pembentukan representasi teks oleh BERT dilakukan melalui beberapa tahapan berikut.

1. **Tokenisasi.** Kalimat dipecah menjadi token menggunakan *WordPiece tokenizer*, lalu ditambahkan token khusus [CLS] di awal dan [SEP] di akhir:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP]].

2. **Padding.** Token [PAD] ditambahkan agar panjang seluruh urutan seragam, misalnya hingga 10 token:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP], [PAD], [PAD], [PAD]].

3. **Attention Mask.** Dibuat *attention mask* untuk membedakan token bermakna dengan token *padding*. Token selain [PAD] diberi nilai 1 dan [PAD] diberi nilai 0:

$$\text{attention mask} = [1, 1, 1, 1, 1, 1, 1, 0, 0, 0].$$

4. **Embedding.** Setiap token diubah menjadi vektor numerik melalui penjumlahan tiga komponen, yaitu *token embedding*, *segment embedding*, dan *position embedding*.

- *Token embedding* merepresentasikan makna setiap token. Setiap token dipetakan ke indeks dalam kosakata model BERT yang digunakan, kemudian diubah menjadi vektor berdimensi 768 menggunakan matriks

$\mathbf{E}_{\text{token}} \in \mathbb{R}^{|\mathcal{V}| \times 768}$, dengan $|\mathcal{V}|$ menyatakan ukuran kosakata model. Sebagai contoh, token pelayanan dipetakan ke indeks $v = 2543$:

$$\mathbf{E}_{\text{token}}[2543] = [0,21, -0,54, 0,83, \dots] \in \mathbb{R}^{768}$$

- *Segment embedding* digunakan untuk membedakan asal kalimat suatu token. Karena penelitian ini hanya menggunakan satu kalimat, seluruh token diberi label Segmen A ($s = 0$):

$$\mathbf{E}_{\text{segment}}[0] = [0,01, 0,01, 0,01, \dots] \in \mathbb{R}^{768}$$

- *Position embedding* memberikan informasi posisi token dalam urutan kalimat. Misalnya, token [CLS] berada pada posisi $p = 0$, sedangkan token pelayanan berada pada posisi $p = 1$:

$$\mathbf{E}_{\text{position}}[1] = [-0,12, 0,34, -0,07, \dots] \in \mathbb{R}^{768}$$

Ketiga komponen tersebut dijumlahkan untuk membentuk representasi akhir token. Untuk token pelayanan dengan $v = 2543$, $s = 0$, dan $p = 1$, diperoleh:

$$\begin{aligned} \mathbf{e}_{\text{pelayanan}} &= \mathbf{E}_{\text{token}}[2543] + \mathbf{E}_{\text{segment}}[0] + \mathbf{E}_{\text{position}}[1] \\ &= [0,21, -0,54, 0,83, \dots] \\ &\quad + [0,01, 0,01, 0,01, \dots] \\ &\quad + [-0,12, 0,34, -0,07, \dots] \\ &= [0,10, -0,19, 0,77, \dots] \in \mathbb{R}^{768} \end{aligned} \tag{3.1}$$

Proses yang sama dilakukan pada seluruh token sehingga diperoleh matriks *embedding* awal $\mathbf{E} \in \mathbb{R}^{10 \times 768}$ sebagai masukan ke lapisan encoder BERT.

5. **Encoding.** Matriks $\mathbf{E}$ diproses melalui 12 lapisan *encoder*. Pada setiap lapisan, mekanisme *multi-head self-attention* menghitung bobot perhatian antar token:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

dengan $d_k = 64$ untuk setiap *head* ($768/12 = 64$), lalu diproses oleh *feed-forward network* dengan koneksi residu dan normalisasi:

$$\mathbf{H}^{(l)} = \text{LayerNorm}\left(\mathbf{H}^{(l-1)} + \text{FFN}\left(\text{MultiHead}\left(\mathbf{H}^{(l-1)}\right)\right)\right)$$

Setelah 12 lapisan, diambil vektor token [CLS] dari keluaran lapisan terakhir sebagai representasi dokumen:

$$\begin{aligned}\mathbf{f} &= \mathbf{H}_{[0,:]}^{(12)} \\ &= [0,31, -0,42, 0,88, \dots] \in \mathbb{R}^{768}\end{aligned}\quad (3.2)$$

Vektor $\mathbf{f}$ ini digunakan sebagai masukan *autoencoder* pada tahap berikutnya, dengan seluruh bobot BERT dibekukan selama pelatihan.

Model BERT yang digunakan memiliki 12 lapisan *encoder* dengan *hidden size* 768. Untuk setiap teks, BERT menghasilkan representasi berukuran $t \times 768$, dengan t adalah panjang maksimum token. Pada penelitian ini digunakan $t = 128$.

Karena proses *clustering* memerlukan representasi berbentuk vektor, digunakan token [CLS] yang merepresentasikan konteks keseluruhan teks. Jika $\mathbf{H} \in \mathbb{R}^{t \times 768}$ adalah keluaran BERT, maka vektor representasi dokumen diperoleh dari token [CLS] sebagai berikut:

$$\mathbf{f} = \mathbf{h}_1 \in \mathbb{R}^{768} \quad (3.3)$$

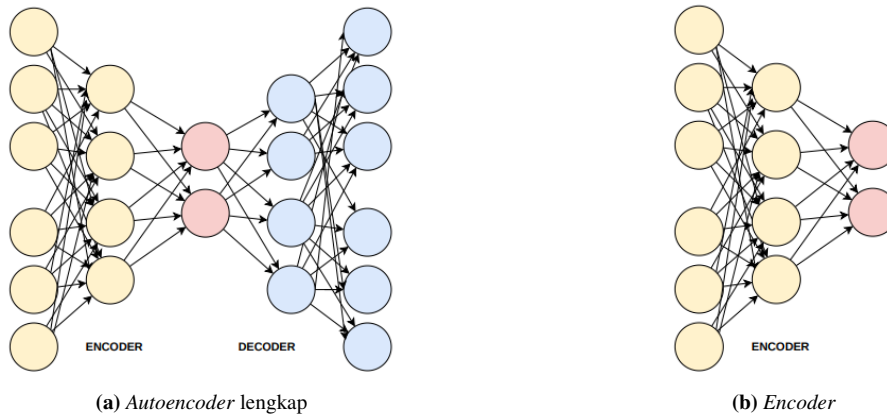
Vektor $\mathbf{f}$ selanjutnya digunakan sebagai masukan bagi *autoencoder*.

3.2 Alur Pelatihan Model *Multitask*

3.2.1 Implementasi *Autoencoder*

Pelatihan *autoencoder* secara langsung pada representasi berdimensi tinggi dapat menyebabkan representasi laten yang kurang stabil serta meningkatkan risiko *overfitting*. Oleh karena itu, sebelum pelatihan *multitask*, *autoencoder* terlebih dahulu dilatih secara *unsupervised* melalui proses *pretraining* sebagai inisialisasi bobot awal.

Gambar 3.3 menunjukkan ilustrasi arsitektur yang digunakan, mulai dari *autoencoder* lengkap hingga komponen *encoder* yang dipertahankan setelah *pretraining*.



Gambar 3.3: Ilustrasi *autoencoder* lengkap (kiri) dan komponen *encoder* setelah *decoder* dilepas (kanan).

Pada tahap *pretraining*, *encoder* dilatih untuk merekonstruksi kembali data *input* dari representasi laten yang dihasilkannya. Fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE) antara vektor *input* asli dan hasil rekonstruksinya:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_i - \hat{\mathbf{f}}_i\|^2 \quad (3.4)$$

dengan N adalah jumlah total sampel, $\mathbf{f}_i$ adalah vektor representasi BERT ke- i , dan $\hat{\mathbf{f}}_i$ adalah hasil rekonstruksinya.

Fungsi aktivasi yang digunakan pada lapisan *encoder* dan *decoder* adalah *Rectified Linear Unit* (ReLU), yang dipilih karena kesederhanaannya, efisiensi komputasi, serta kemampuannya mengurangi permasalahan *vanishing gradient* pada jaringan dalam.

Setelah *pretraining* selesai, bobot *encoder* digunakan sebagai inisialisasi untuk pelatihan *multitask*. Pada tahap ini *encoder* dipertahankan dan dioptimalkan bersama kedua *head*, sedangkan *decoder* tidak lagi digunakan. Representasi laten $\mathbf{z} = f_{\theta}(\mathbf{f})$ yang dihasilkan *encoder* digunakan bersama oleh *sentiment head* dan *clustering head*.

3.2.2 Deep Embedded Clustering (DEC)

DEC bertujuan memperbarui representasi laten dan menyesuaikan posisi *centroid* agar sesuai dengan struktur distribusi data pada ruang laten (Xie, Girshick, dan Farhadi, 2016). Berbeda dengan *hard assignment* yang menetapkan setiap sampel ke satu kluster, DEC menggunakan distribusi probabilistik sehingga setiap sampel memiliki derajat keanggotaan terhadap seluruh kluster.

Proses DEC dimulai dengan inisialisasi *centroid* menggunakan K-Means pada

representasi laten awal hasil *pretraining encoder*. Distribusi *soft assignment* q_{ij} untuk setiap data $\mathbf{z}_i$ terhadap *centroid* $\boldsymbol{\mu}_j$ kemudian dihitung menggunakan distribusi Student's t :

$$q_{ij} = \frac{\left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_j\|^2\right)^{-1}}{\sum_{j'=1}^K \left(1 + \|\mathbf{z}_i - \boldsymbol{\mu}_{j'}\|^2\right)^{-1}} \quad (3.5)$$

dengan q_{ij} menyatakan probabilitas sampel i termasuk klaster j , dan K adalah jumlah klaster.

DEC kemudian mendefinisikan distribusi target p_{ij} yang lebih terkonsentrasi (*sharpened*) agar setiap sampel lebih condong ke satu klaster. Misalkan $f_j = \sum_i q_{ij}$ adalah frekuensi *soft* klaster j , maka:

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'=1}^K q_{ij'}^2 / f_{j'}} \quad (3.6)$$

Pembagian dengan f_j mencegah klaster besar mendominasi proses optimasi, sedangkan kuadrat pada q_{ij} membuat distribusi lebih terkonsentrasi pada satu klaster. Optimasi dilakukan dengan meminimalkan *Kullback-Leibler Divergence* antara distribusi target P dan distribusi *soft assignment* Q :

$$\mathcal{L}_{\text{KLD}} = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3.7)$$

$\mathcal{L}_{\text{KLD}}$ digunakan sebagai dasar untuk memperbarui parameter *encoder* dan posisi *centroid* menggunakan optimizer SGD secara iteratif. Pada setiap *update interval*, label klaster hasil iterasi sebelumnya dibandingkan dengan label terbaru. Pelatihan dihentikan apabila perubahan label klaster (Δ_{label}) berada di bawah batas toleransi τ . Tabel 3.1 menyajikan prosedur lengkap DEC.

Tabel 3.1: Algoritma *Deep Embedded Clustering* (DEC)

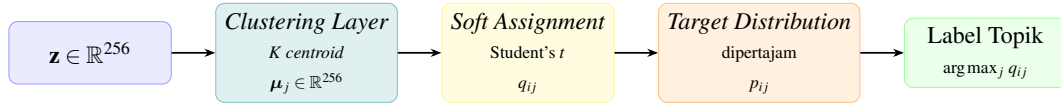
Algoritma <i>Deep Embedded Clustering</i> (DEC)	
INPUT	Representasi laten $\{\mathbf{z}_i\}_{i=1}^N \in \mathbb{R}^{256}$ hasil <i>pretraining encoder</i> ; jumlah kluster K ; <i>batch size</i> b ; batas toleransi konvergensi τ ; jumlah iterasi maksimum $T_{\max}$
OUTPUT	Posisi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$; label kluster $\{y_{\text{cluster},i}\}$
1.	Inisialisasi <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$ menggunakan K-Means pada $\{\mathbf{z}_i\}$
2.	FOR iter = 1 hingga $T_{\max}$ 2.1 Simpan label kluster sebelumnya: $y_{\text{prev}} \leftarrow \arg \max_j q_{ij}$ 2.2 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.5) 2.3 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.6) 2.4 Perbarui parameter <i>encoder</i> dan <i>centroid</i> dengan meminimalkan $\mathcal{L}_{\text{KLD}}(P, Q)$ 2.5 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}_{[y_{\text{prev},i} \neq \arg \max_j q_{ij}]}$ 2.6 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Kembalikan <i>centroid</i> $\{\boldsymbol{\mu}_j\}$ dan label kluster $\{y_{\text{cluster},i}\}$

3.2.3 Pelatihan *Multitask*: JST-DEC

Pendekatan *multitask* memungkinkan representasi laten dipelajari secara bersama untuk dua tugas, yaitu klasifikasi sentimen dan klusterisasi topik. Melalui pendekatan ini, proses pembelajaran representasi mempertimbangkan informasi sentimen dan struktur topik secara simultan dalam satu ruang laten, berbeda dengan pendekatan yang mengoptimalkan kedua tugas secara terpisah. Kombinasi DEC dengan *sentiment head* dalam kerangka *multitask* ini selanjutnya disebut JST-DEC (*Joint Sentiment-Topic Deep Embedded Clustering*).

3.2.3.1 *Clustering Head*

Vektor laten $\mathbf{z}$ yang dihasilkan *encoder* diteruskan ke *clustering head* untuk pendeteksian topik. *Clustering head* mengadopsi mekanisme DEC sehingga setiap sampel memiliki derajat keanggotaan probabilistik terhadap seluruh kluster, bukan penetapan ke satu kluster secara langsung. Gambar 3.4 mengilustrasikan arsitekturnya.

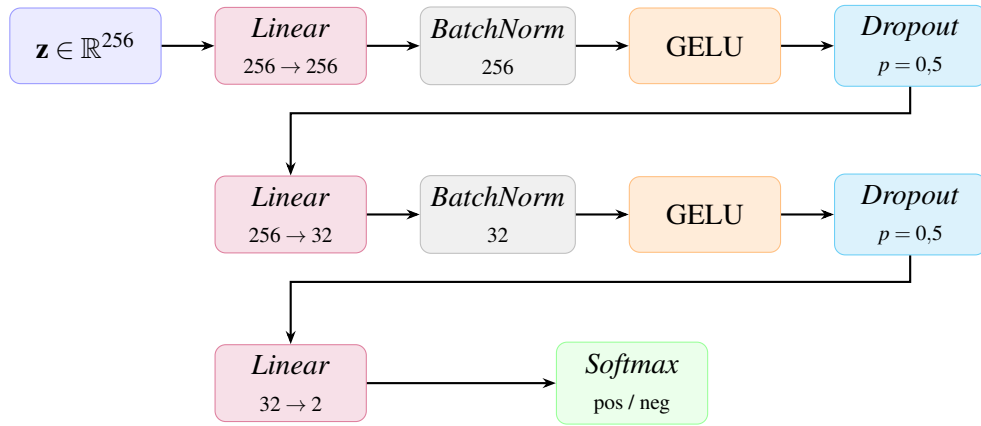


Gambar 3.4: Arsitektur *clustering head* berbasis DEC.

Clustering head terdiri atas satu lapisan kluster yang menyimpan K *centroid* $\mu_j \in \mathbb{R}^{256}$. Untuk setiap vektor laten $\mathbf{z}_i$, lapisan ini menghitung distribusi *soft assignment* q_{ij} terhadap seluruh *centroid* menggunakan distribusi Student's t sebagaimana didefinisikan pada Persamaan 3.5. Distribusi tersebut kemudian dipertajam menjadi distribusi target p_{ij} (Persamaan 3.6), dan label topik akhir diperoleh melalui $\arg \max_j q_{ij}$.

3.2.3.2 Analisis Sentimen Menggunakan *Sentiment Head*

Vektor laten $\mathbf{z} \in \mathbb{R}^{256}$ yang dihasilkan *encoder* diteruskan ke *sentiment head* untuk diklasifikasikan ke salah satu dari dua kelas sentimen. Arsitektur *sentiment head* terdiri atas tiga lapisan linear bertingkat dengan dimensi $256 \rightarrow 256 \rightarrow 32 \rightarrow 2$. Setiap lapisan tersembunyi dilengkapi dengan *batch normalization*, fungsi aktivasi *Gaussian Error Linear Unit* (GELU), dan *dropout* dengan probabilitas $p = 0,5$. Lapisan keluaran berdimensi dua diikuti fungsi *softmax* untuk menghasilkan distribusi probabilitas antarkelas. Gambar 3.5 mengilustrasikan arsitektur lengkapnya.



Gambar 3.5: Arsitektur *sentiment head*.

Proses klasifikasi dirumuskan sebagai:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c) \quad (3.8)$$

dengan $\mathbf{W}_c$ dan $\mathbf{b}_c$ masing-masing adalah bobot dan bias pada lapisan keluaran. Fungsi *softmax* mengubah nilai logit menjadi distribusi probabilitas, di mana setiap elemen $\hat{y}_i \in [0, 1]$ dan $\sum_i \hat{y}_i = 1$.

Selama pelatihan, prediksi $\hat{\mathbf{y}}$ dibandingkan dengan label sebenarnya $\mathbf{y}$ menggunakan fungsi *loss Categorical Cross-Entropy*:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^N \sum_{j=1}^2 y_{ij} \log \hat{y}_{ij} \quad (3.9)$$

dengan N adalah jumlah sampel, y_{ij} adalah label aktual dalam bentuk *one-hot* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model.

3.2.3.3 Pelatihan *Joint* dengan *Multitask Loss*

Total *loss* yang dioptimalkan adalah kombinasi berbobot antara $\mathcal{L}_{\text{KLD}}$ dari *clustering head* dan $\mathcal{L}_{\text{CE}}$ dari *sentiment head*:

$$\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) \quad (3.10)$$

dengan γ mengontrol kontribusi informasi pengelompokan dan η mengontrol kontribusi informasi sentimen.

Gradien dari $\mathcal{L}_{\text{KLD}}$ mengarahkan *encoder* untuk membentuk representasi yang terstruktur berdasarkan kluster, sementara gradien dari $\mathcal{L}_{\text{CE}}$ mengarahkan representasi laten agar lebih mampu membedakan label sentimen. Kedua sinyal gradien dioptimalkan secara simultan sehingga representasi laten diharapkan mampu mempertahankan struktur topik tanpa menghilangkan informasi sentimen.

Nilai γ dan η ditentukan melalui pencarian *hyperparameter* pada data validasi. Jika $\gamma \gg \eta$, model cenderung mengutamakan struktur kluster; sebaliknya jika $\eta \gg \gamma$, representasi laten lebih diarahkan oleh informasi sentimen.

3.2.3.4 Algoritma JST-DEC

Tabel 3.2 menyajikan prosedur lengkap pelatihan JST-DEC. Algoritma ini mengasumsikan *encoder* telah diinisialisasi melalui *pretraining autoencoder* dan *centroid* telah diinisialisasi menggunakan K-Means sebagaimana dijelaskan pada Subseksi 3.2.2.

Tabel 3.2: Algoritma JST-DEC (*Joint Sentiment-Topic Deep Embedded Clustering*)

Algoritma JST-DEC	
INPUT	Himpunan vektor BERT $\{\mathbf{f}_i\}_{i=1}^N \in \mathbb{R}^{768}$; label sentimen $\{y_{\text{sentiment},i}\}$; <i>encoder</i> f_θ dan <i>centroid</i> $\{\boldsymbol{\mu}_j\}$ hasil inisialisasi DEC; parameter bobot <i>loss</i> γ, η ; <i>batch size</i> b ; batas toleransi τ ; $T_{\max}$
OUTPUT	Representasi laten $\{\mathbf{z}_i\}$; label kluster $\{y_{\text{cluster},i}\}$; label sentimen $\{\hat{y}_{\text{sentiment},i}\}$
1.	Hitung $q_{ij}^{(0)}$ dari <i>centroid</i> inisialisasi DEC; tetapkan $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}^{(0)}$
2.	FOR iter = 1 hingga $T_{\max}$ 2.1 Simpan label kluster sebelumnya: $y_{\text{prev}} \leftarrow y_{\text{cluster}}$ 2.2 Proyeksikan data ke ruang laten: $\mathbf{z}_i = f_\theta(\mathbf{f}_i)$ 2.3 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t (Persamaan 3.5) 2.4 Perbarui label kluster: $y_{\text{cluster},i} \leftarrow \arg \max_j q_{ij}$ 2.5 Hitung distribusi target p_{ij} berdasarkan q_{ij} (Persamaan 3.6) 2.6 Hitung prediksi sentimen: $\hat{y}_i = \text{softmax}(\mathbf{W}_c \mathbf{z}_i + \mathbf{b}_c)$ 2.7 Hitung <i>joint loss</i>: $\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}})$ 2.8 Perbarui seluruh parameter model ($f_\theta, \boldsymbol{\mu}_j, \mathbf{W}_c, \mathbf{b}_c$) menggunakan optimizer SGD 2.9 Hitung perubahan label: $\Delta_{\text{label}} = \frac{1}{N} \sum_i \mathbf{1}[y_{\text{prev},i} \neq y_{\text{cluster},i}]$ 2.10 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
3.	END FOR
4.	Simpan bobot terbaik model berdasarkan metrik validasi
OUTPUT:	Representasi laten $\{\mathbf{z}_i\}$, label kluster $\{y_{\text{cluster},i}\}$, label sentimen $\{\hat{y}_{\text{sentiment},i}\}$

3.3 Interpretasi Model dengan Analisis Atribusi Token

Penelitian ini menerapkan dua metode interpretasi *post-hoc*, yaitu *occlusion-based token attribution* pada level token dan *Integrated Gradients* pada level dimensi *embedding*. Keduanya diterapkan setelah pelatihan selesai menggunakan bobot model terbaik.

3.3.1 Implementasi *Occlusion-based Token Attribution* dan Agregasi per Kluster

Teks *input* $\mathbf{x}$ diproses oleh BERT untuk menghasilkan representasi $[\text{CLS}]$, yang kemudian diteruskan ke *sentiment head* guna memperoleh probabilitas *baseline*

$f_c(\mathbf{x})$ untuk kelas c . Berikutnya, setiap token x_i dalam teks, kecuali token-token spesial [CLS], [SEP], dan [PAD], digantikan dengan [MASK] sehingga menghasilkan varian teks $\mathbf{x}_{\setminus i}$. Varian tersebut kemudian diproses kembali melalui BERT dan *sentence head* untuk memperoleh probabilitas yang diperbarui, $f_c(\mathbf{x}_{\setminus i})$. Skor atribusi token x_i terhadap kelas c selanjutnya dihitung dengan:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}) \quad (3.11)$$

Nilai positif mengindikasikan bahwa token x_i berkontribusi terhadap prediksi kelas c , nilai negatif mengindikasikan kontribusi yang berlawanan arah, sedangkan nilai yang mendekati nol mengindikasikan bahwa token tidak memberikan pengaruh yang berarti.

Skor atribusi kemudian diagregasi per klaster untuk melihat pola linguistik pada level topik. Misalkan $C_k = \{i : z_i = k\}$ adalah himpunan sampel pada klaster k , dengan z_i adalah label klaster sampel ke- i . Skor atribusi rata-rata token w pada klaster k didefinisikan sebagai:

$$\overline{\text{Attr}}(w, k, c) = \frac{1}{|C_k|} \sum_{i \in C_k} \text{Attr}(x_i^{(w)}, c) \quad (3.12)$$

dengan $x_i^{(w)}$ merujuk pada posisi token w dalam sampel ke- i dan $|C_k|$ adalah jumlah sampel dalam klaster k . Hanya token yang muncul pada minimal dua sampel dalam klaster yang sama yang diikutsertakan, yaitu:

$$\mathcal{V}_k = \left\{ w \mid \sum_{i \in C_k} \mathbf{1}[w \in \mathbf{x}_i] \geq 2 \right\} \quad (3.13)$$

Token dalam $\mathcal{V}_k$ diurutkan berdasarkan nilai absolut skor atribusi rata-rata untuk memperoleh daftar R token paling berpengaruh pada klaster k :

$$\text{TopTokens}(k, c) = \arg \max_{w \in \mathcal{V}_k} \text{top-}R \left| \overline{\text{Attr}}(w, k, c) \right| \quad (3.14)$$

Dalam implementasinya, kosakata yang dievaluasi dibatasi pada *top-N* kata berdasarkan TF-IDF per klaster, sehingga skor atribusi hanya dihitung untuk kata-kata yang khas di klaster tersebut.

3.3.2 Implementasi *Integrated Gradients*

Integrated Gradients diterapkan pada level dimensi *embedding* untuk mengidentifikasi dimensi yang paling berpengaruh terhadap prediksi sentimen. *Baseline*

yang digunakan adalah vektor nol $\mathbf{x}' = \mathbf{0} \in \mathbb{R}^d$, dengan jumlah langkah interpolasi $m = 50$ sebagaimana direkomendasikan oleh Sundararajan, Taly, dan Yan (2017). Atribusi IG untuk setiap dimensi *embedding* diaproksimasi menggunakan *Riemann sum*:

$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}'))}{\partial x_i} \times \frac{1}{m} \quad (3.15)$$

dengan x_i adalah nilai dimensi ke- i dari *embedding input*, x'_i adalah nilai *baseline* pada dimensi yang sama, dan $\frac{\partial f}{\partial x_i}$ adalah gradien prediksi model terhadap dimensi ke- i . Atribusi IG kemudian diagregasi per klaster:

$$\overline{\text{IG}}_i(k) = \frac{1}{|C_k|} \sum_{j \in C_k} |\text{IG}_i(\mathbf{x}_j)| \quad (3.16)$$

dengan $\overline{\text{IG}}_i(k)$ adalah rata-rata nilai absolut atribusi IG dimensi ke- i pada klaster k , dan $\mathbf{x}_j$ adalah *embedding* sampel ke- j dalam klaster k . Hasil agregasi ini divisualisasikan dalam bentuk *heatmap* untuk memudahkan perbandingan antarklaster.

3.4 Evaluasi Topik

Evaluasi kualitas topik dilakukan menggunakan tiga metrik kuantitatif, yaitu *topic coherence*, *topic diversity*, dan *topic coverage*. Ketiga metrik dihitung langsung dari teks asli dan hasil pengelompokan klaster tanpa bergantung pada model eksternal.

3.4.1 Topic Coherence Berbasis NPMI

Topic coherence mengukur sejauh mana kata-kata teratas suatu topik sering muncul bersama dalam korpus. Metrik yang digunakan adalah *Normalized Pointwise Mutual Information* (NPMI) untuk mengukur ko-okurens antar pasangan kata dalam klaster yang sama.

Misalkan $\mathcal{D}_k$ adalah himpunan teks dalam klaster k dan $\{w_1, w_2, \dots, w_N\}$ adalah N kata teratas dari klaster tersebut berdasarkan frekuensi kemunculan setelah pemfilteran *stop words*. Untuk setiap pasangan kata (w_i, w_j) dengan $i < j$, probabilitas kemunculan masing-masing kata dan ko-okurensnya didefinisikan sebagai:

$$p(w_i) = \frac{|\{d \in \mathcal{D}_k : w_i \in d\}|}{|\mathcal{D}_k|}, \quad (3.17)$$

$$p(w_j) = \frac{|\{d \in \mathcal{D}_k : w_j \in d\}|}{|\mathcal{D}_k|}, \quad (3.18)$$

$$p(w_i, w_j) = \frac{|\{d \in \mathcal{D}_k : w_i \in d \text{ dan } w_j \in d\}|}{|\mathcal{D}_k|}. \quad (3.19)$$

Nilai *Pointwise Mutual Information* (PMI) antara dua kata dihitung sebagai:

$$\text{PMI}(w_i, w_j) = \log \frac{p(w_i, w_j) + \epsilon}{p(w_i) \cdot p(w_j) + \epsilon} \quad (3.20)$$

dengan $\epsilon = 10^{-10}$ untuk menghindari pembagian dengan nol. Selanjutnya, PMI dinormalisasi menggunakan probabilitas ko-okurens sebagai pembagi:

$$\text{NPMI}(w_i, w_j) = \frac{\text{PMI}(w_i, w_j)}{-\log(p(w_i, w_j) + \epsilon)} \quad (3.21)$$

Nilai NPMI berada pada rentang $[-1, 1]$: mendekati 1 berarti ko-okurens tinggi, 0 berarti independen, dan negatif berarti kata-kata jarang muncul bersama. Skor *topic coherence* untuk kluster k adalah rata-rata NPMI atas semua pasangan kata teratas:

$$\text{TC}_k = \frac{1}{\binom{N}{2}} \sum_{j=2}^N \sum_{i=1}^{j-1} \text{NPMI}(w_i, w_j) \quad (3.22)$$

Skor koherensi keseluruhan diperoleh dengan merata-ratakan TC_k di seluruh kluster:

$$\overline{\text{TC}} = \frac{1}{K} \sum_{k=1}^K \text{TC}_k \quad (3.23)$$

Metrik ini hanya dihitung untuk kluster dengan setidaknya dua kata teratas yang unik; kluster yang tidak memenuhi syarat diberi nilai koherensi 0.

3.4.2 Topic Diversity

Topic diversity mengukur seberapa berbeda kosakata antarkelas. Misalkan $\mathcal{W}_k$ adalah himpunan N kata teratas dari kluster k , himpunan kata unik di seluruh kluster $\mathcal{U} = \bigcup_{k=1}^K \mathcal{W}_k$, dan total kata termasuk duplikat adalah $\sum_{k=1}^K |\mathcal{W}_k|$. *Topic diversity* didefinisikan sebagai:

$$TD = \frac{|\mathcal{U}|}{\sum_{k=1}^K |\mathcal{W}_k|} \quad (3.24)$$

Nilai $TD = 1$ berarti seluruh kata teratas antarkelas unik, sedangkan nilai rendah menunjukkan banyak tumpang tindih kosakata.

BAB 4

HASIL DAN SIMULASI DAN PEMBAHASAN

Bab ini membahas hasil simulasi dan analisis kinerja model *Joint Sentiment-Topic* berbasis *multitask learning* yang diusulkan. Data yang digunakan berasal dari dataset IMDB dan Yelp yang telah melalui tahap prapengolahan dan pembagian data. Kinerja model dievaluasi dari dua aspek, yaitu analisis sentimen dan pendeteksian topik. Pada analisis sentimen, performa diukur menggunakan metrik *accuracy*, *precision*, *recall*, dan *F1-score*. Pada pendeteksian topik, kualitas topik dinilai menggunakan metrik *Topic Coherence* berbasis NPMI dan *Topic Diversity*. Hasil interpretasi model melalui analisis atribusi token juga dipaparkan untuk menggambarkan karakteristik linguistik tiap kluster topik.

4.1 Spesifikasi Mesin dan Perangkat Lunak

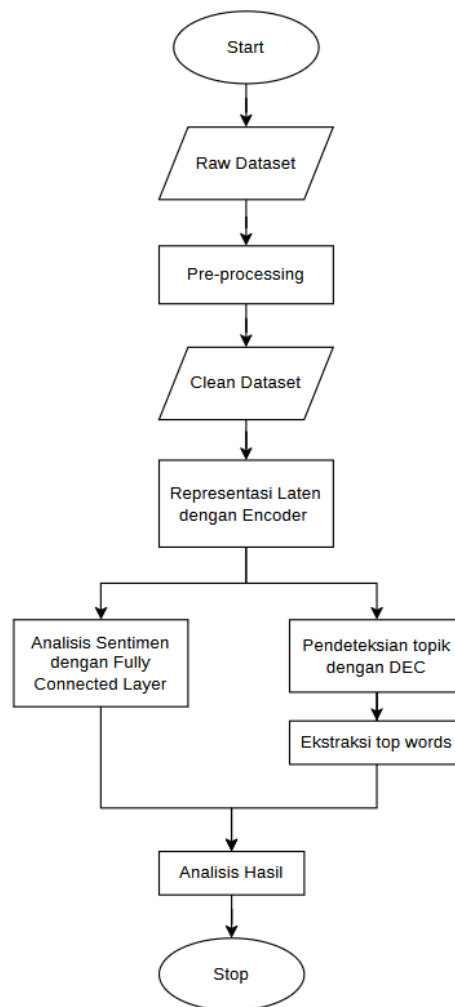
Spesifikasi mesin dan perangkat lunak yang digunakan dalam proses simulasi pada penelitian ini dapat dilihat pada Tabel 4.1.

Tabel 4.1: Spesifikasi mesin dan perangkat lunak

Mesin	
<i>Central Processing Unit</i> (CPU)	Apple M1
Memory	8 GB
<i>Graphic Processing Unit</i> (GPU)	Apple M1 GPU (8-core)
Perangkat Lunak	
Perangkat Lunak Pemrograman	Google Colaboratory Kaggle Notebooks Visual Studio Code
Bahasa Pemrograman	Python 3.13.3
Pustaka yang Digunakan	- os, re, csv, json, glob - numpy, pandas - PyTorch (torch) - scikit-learn, scipy - transformers - datasets - tqdm - seaborn

4.2 Tahapan Umum Simulasi

Tahapan umum simulasi pada penelitian ini digambarkan dalam diagram alur pada Gambar 4.1.



Gambar 4.1: Diagram alur simulasi penelitian.

Proses dimulai dengan pengumpulan data ulasan yang kemudian melewati tahap prapengolahan untuk menghasilkan data yang bersih. Data tersebut selanjutnya diproses oleh BERT yang digunakan secara *feature-based* dengan bobot yang dibekukan, menghasilkan representasi vektor [CLS] berdimensi 768 untuk setiap dokumen. Representasi ini kemudian menjadi masukan bagi *autoencoder* untuk menghasilkan representasi laten berdimensi lebih rendah.

Representasi laten yang dihasilkan digunakan secara bersamaan oleh dua jalur tugas, yaitu analisis sentimen melalui *sentiment head* dan pendeteksian topik melalui *Deep Embedded Clustering* (DEC). Setelah pelatihan *joint* selesai, kinerja dari kedua tugas dievaluasi dengan metrik yang sesuai.

4.3 Data

Subbab ini menjelaskan data yang digunakan dalam penelitian, meliputi sumber data, karakteristik dataset, serta tahapan persiapan data sebelum digunakan dalam pelatihan model. Secara umum, proses pengolahan data dimulai dari pengumpulan dataset ulasan, lalu dilanjutkan dengan prapengolahan teks, pembentukan representasi menggunakan BERT, dan pembagian data.

4.3.1 Pengumpulan Data

Penelitian ini menggunakan dua dataset ulasan berbahasa Inggris, yaitu IMDB Movie Reviews dan Yelp Review Dataset. Kedua dataset dipilih karena umum digunakan sebagai *benchmark* pada penelitian analisis sentimen dan pendeteksian topik, serta keduanya memiliki label sentimen biner.

Dataset IMDB Movie Reviews terdiri dari 30.000 ulasan film yang diunggah oleh (Maas *et al.*, 2011b) dan tersedia secara publik melalui platform Hugging Face dalam format Apache Arrow. Distribusi label sentimen pada dataset ini relatif seimbang antara kelas positif dan negatif, sehingga cocok untuk tugas klasifikasi biner.

Yelp Review Dataset memuat 50.000 ulasan pengguna terhadap berbagai jenis layanan dan bisnis, termasuk restoran dan usaha ritel. Dataset ini tersedia secara publik melalui platform Hugging Face yang diunggah oleh (X. Zhang, J. Zhao, dan LeCun, 2015b) dalam format Apache Arrow dengan distribusi label sentimen yang seimbang. Dalam penelitian ini, jumlah data Yelp dibatasi hingga 30.000 sampel agar proporsional dengan jumlah data IMDB.

Kedua dataset memiliki label sentimen biner, dengan nilai 1 untuk sentimen positif dan nilai 0 untuk sentimen negatif. Beberapa cuplikan ulasan dari masing-masing dataset ditampilkan pada Tabel 4.2 dan Tabel 4.3.

Tabel 4.2: Cuplikan data ulasan IMDB (Maas *et al.*, 2011b).

No	Text	Label
1	An absolute masterpiece. The storytelling is compelling from start to finish, and the performances are nothing short of extraordinary. Highly recommended to anyone who appreciates great cinema.	1
2	A complete waste of time. The plot makes no sense, the acting is wooden, and the ending is thoroughly unsatisfying. Save yourself two hours and skip this one.	0
⋮	⋮	⋮
30.000	The film captures a quiet kind of grief beautifully. Not much happens on the surface, but every scene carries emotional weight. A slow burn that rewards patient viewers.	1

Tabel 4.3: Cuplikan data ulasan Yelp (X. Zhang, J. Zhao, dan LeCun, 2015b).

No	Text	Label
1	Hands down the best ramen I have had outside of Japan. The broth is rich and complex, the noodles perfectly chewy. The staff were attentive without being intrusive. Will absolutely return.	1
2	Ordered delivery and the food arrived nearly an hour late and completely cold. When I called to complain, the staff were dismissive. Would not order from here again.	0
⋮	⋮	⋮
30.000	Decent enough for a quick lunch but nothing memorable. The portions are on the smaller side for the price, and the ambiance is a bit too noisy to hold a conversation comfortably.	0

4.3.2 Prapengolahan Data

Tahap prapengolahan bertujuan membersihkan teks dari elemen yang tidak mengandung informasi bahasa sebelum diproses oleh BERT. Karena BERT merupakan model berbasis *transformer* yang menangkap hubungan kontekstual antar-token, penelitian ini menerapkan prapengolahan yang minimalis. Tahapan seperti *stemming*, penghapusan *stop words*, dan normalisasi tanda baca tidak diterapkan, karena kata-kata fungsional seperti *stop words* masih mengandung informasi kontekstual yang relevan bagi model.

Selain itu, BERT menggunakan tokenisasi berbasis WordPiece yang mampu memecah kata di luar kosakata menjadi unit *subword*, sehingga normalisasi kata secara manual tidak diperlukan. Prapengolahan pada penelitian ini difokuskan

pada pembersihan elemen non-linguistik tanpa menghilangkan kandungan informasi dalam teks. Tahapan yang diterapkan adalah sebagai berikut.

1. *Casefolding*, yaitu mengubah seluruh teks menjadi huruf kecil untuk menghindari perbedaan representasi akibat variasi kapitalisasi.
2. Penghapusan tag HTML, yaitu menghapus elemen HTML seperti `<br />` dan `<b>` yang tidak memiliki kandungan bahasa.
3. Penghapusan URL, yaitu menghapus tautan dengan pola `http://...` atau `www....` karena tidak membawa informasi sentimen maupun topik.
4. Penghapusan karakter non-ASCII, yaitu menghapus emoji, simbol unicode, dan karakter khusus yang dapat mengganggu proses tokenisasi.
5. Normalisasi *whitespace*, yaitu menyeragamkan spasi berlebih, karakter tab, dan baris baru menjadi satu spasi tunggal.

Hasil dari tahap ini adalah teks yang bersih dari elemen non-linguistik namun tetap mempertahankan struktur kalimat aslinya. Perbandingan teks sebelum dan sesudah prapengolahan dapat dilihat pada Tabel 4.4.

Tabel 4.4: Contoh hasil prapengolahan pada dataset IMDB dan Yelp.

Sebelum Prapengolahan	Sesudah Prapengolahan
<i>IMDB</i>	
An absolute MASTERPIECE. The storytelling is compelling. <code>
</code> Highly recommended to anyone who appreciates great cinema!	an absolute masterpiece. the storytelling is compelling. highly recommended to anyone who appreciates great cinema!
A complete waste of time. <code>
</code> Read the full review at http://www.filmcritic.org/reviews/2024	a complete waste of time. read the full review at
<i>Yelp</i>	
Hands down the BEST ramen outside of Japan :) The broth is incredible! Check the menu at www.ramenhouse.com	hands down the best ramen outside of japan the broth is incredible! check the menu at
Ordered delivery & the food arrived cold after 1hr. Staff were rude when I called :(Would NOT recommend.	ordered delivery the food arrived cold after 1hr. staff were rude when i called would not recommend.

Tabel 4.4 menunjukkan bahwa perubahan yang dilakukan terbatas pada penghapusan elemen non-linguistik dan penyeragaman format teks, bukan penyederhanaan isi kalimat. Dengan demikian, kandungan kontekstual dalam ulasan tetap terjaga untuk tahap ekstraksi fitur menggunakan BERT.

4.3.3 Representasi Teks

Setelah prapengolahan, teks diubah menjadi representasi numerik melalui dua tahap, yaitu tokenisasi dan ekstraksi fitur menggunakan BERT. Representasi yang dihasilkan menjadi masukan bagi *autoencoder* pada tahap pelatihan berikutnya.

Tokenisasi dilakukan menggunakan tokenizer WordPiece dari model BERT *pre-trained*. Setiap teks dikonversi menjadi urutan *token ID* dengan panjang maksimum $t = 128$ token. Teks yang melebihi batas ini akan dipotong (*truncation*), sedangkan teks yang lebih pendek akan ditambahkan token [PAD] bernilai 0 agar panjang urutan seragam. Contoh hasil tokenisasi ditampilkan pada Tabel 4.5.

Tabel 4.5: Contoh proses tokenisasi menggunakan WordPiece BERT.

Teks (hasil prapengolahan)	Hasil tokenisasi (<i>token ID</i>)
an absolute masterpiece the storytelling is compelling	[101, 2019, 7927, 4902, 1996, 4129, 2003, 17075, 102, 0, ..., 0]

Urutan *token ID* tersebut selanjutnya diproses oleh BERT untuk menghasilkan representasi kontekstual. Model yang digunakan adalah BERT *pre-trained* dengan *hidden size* 768 dan bobot yang dibekukan sepenuhnya (*frozen*). Dari keluaran BERT berupa matriks berukuran $t \times 768$, diambil baris pertama yang bersesuaian dengan token [CLS] sebagai representasi dokumen:

$$\mathbf{f} = \mathbf{h}_1 \in \mathbb{R}^{768} \quad (4.1)$$

dengan $\mathbf{h}_1$ adalah vektor keluaran BERT pada posisi token [CLS]. Vektor $\mathbf{f}$ ini merepresentasikan konteks global dokumen dan digunakan sebagai masukan bagi *autoencoder* pada tahap berikutnya. Contoh ilustrasi perubahan representasi dari *token ID* ke vektor BERT ditampilkan pada Tabel 4.6.

Tabel 4.6: Contoh representasi vektor [CLS] hasil ekstraksi BERT.

<i>Token ID</i>	Representasi [CLS] $\mathbf{f} \in \mathbb{R}^{768}$
[101, 2019, 7927, 4902, ...]	[0.31, -0.17, 0.08, ..., -0.22]

4.3.4 Pembagian Data

Setelah prapengolahan, data dari kedua dataset dibagi menjadi data latih (*training set*) dan data uji (*test set*) dengan proporsi 80% untuk data latih dan 20% untuk

data uji, mengacu pada skema yang digunakan oleh (Gowandi, Murfi, dan Nurrohmah, 2021). Pembagian ini dilakukan agar proses pelatihan dan evaluasi model dapat dilakukan secara terpisah.

Data latih digunakan sebagai masukan dalam proses optimasi parameter model, baik pada komponen analisis sentimen maupun pendeteksian topik. Data uji digunakan untuk mengukur performa model pada data yang belum pernah dilihat selama pelatihan.

4.4 Implementasi Model

Implementasi pada penelitian ini mencakup tiga model, yaitu JST-DEC sebagai model yang diusulkan, NN-BERT sebagai model pembanding untuk analisis sentimen, dan BERTopic sebagai model pembanding untuk pendeteksian topik. JST-DEC dilatih melalui dua tahap, yaitu pra-pelatihan *autoencoder* dan *joint training*. Seluruh model diimplementasikan menggunakan Python dengan pustaka sebagaimana tercantum pada Tabel 4.1.

4.4.1 Konfigurasi JST-DEC

Konfigurasi JST-DEC mencakup parameter pada tahap pra-pelatihan *autoencoder* dan pembentukan kluster menggunakan DEC. Konfigurasi masing-masing komponen disajikan pada Tabel 4.7 dan Tabel 4.8.

Tabel 4.7: Konfigurasi pra-pelatihan *autoencoder*

Parameter	Nilai
Arsitektur	FC, simetris
Dimensi lapisan	768→2022→2022→256
Dimensi laten	256
Fungsi aktivasi	ReLU
Fungsi <i>loss</i>	MSE
<i>Optimizer</i>	Adam
<i>Learning rate</i>	2×10^{-3}
<i>Batch size</i>	128
Epoch	30

Tabel 4.8: Konfigurasi komponen DEC

Parameter	Nilai
Lapisan klaster	1
Dimensi ruang klaster	256
Jumlah klaster (K)	30, 50, 80
<i>Soft assignment</i>	Student- t
Derajat kebebasan (v)	1
<i>Loss clustering</i>	KL Divergence
Inisialisasi centroid	k -means

Dimensi masukan sebesar 768 mengikuti ukuran vektor [CLS] yang dihasilkan oleh BERT. Representasi tersebut kemudian direduksi ke ruang laten berdimensi 256 untuk menghasilkan fitur yang lebih ringkas sebelum proses klasterisasi. Pada tahap DEC, centroid klaster diinisialisasi menggunakan metode k -means, sedangkan jumlah klaster (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 klaster, guna menganalisis pengaruh jumlah topik terhadap kualitas hasil klasterisasi.

4.4.2 Konfigurasi *Joint Training*

Setelah tahap *pretraining*, JST-DEC dioptimasi melalui proses *joint training* untuk menjalankan dua tugas secara simultan, yaitu klasifikasi sentimen dan pembentukan klaster topik. Fungsi objektif yang digunakan didefinisikan sebagai

$$\mathcal{L} = \eta \mathcal{L}_{\text{sent}} + \gamma \mathcal{L}_{\text{clust}}.$$

Konfigurasi pelatihan yang digunakan pada tahap *joint training* ditunjukkan pada Tabel 4.9.

Tabel 4.9: Konfigurasi *joint training* JST-DEC

Parameter	Nilai
<i>Learning rate</i>	5×10^{-5}
<i>Batch size</i>	256
Iterasi maksimum	200
<i>Update interval</i>	20
Toleransi (τ)	10^{-4}
Bobot sentimen (η)	0,5
Bobot klaster (γ)	0,5
<i>Optimizer</i>	SGD
Jumlah topik (K)	30, 50, dan 80
<i>Seed</i>	42

Nilai η dan γ ditetapkan sebesar 0,5 untuk memberikan kontribusi yang seimbang antara tugas klasifikasi sentimen dan klasterisasi topik selama proses pelatihan. Sementara itu, jumlah topik (K) dievaluasi pada tiga skenario, yaitu 30, 50, dan 80 topik, untuk mengidentifikasi konfigurasi yang memberikan hasil terbaik.

4.4.3 Konfigurasi Model Pembandingan

Penelitian ini menggunakan dua model pembandingan untuk mengevaluasi kinerja JST-DEC pada masing-masing tugas. NN-BERT digunakan sebagai model pembandingan untuk analisis sentimen karena memanfaatkan representasi BERT yang sama tanpa mekanisme pembelajaran multitugas. Sementara itu, BERTopic digunakan sebagai pembandingan pada pendeteksian topik karena merupakan salah satu pendekatan berbasis *transformer* yang banyak digunakan untuk *topic modeling*.

4.4.3.1 NN-BERT

NN-BERT digunakan sebagai model dasar (*baseline*) untuk tugas analisis sentimen. Representasi [CLS] yang dihasilkan oleh BERT digunakan sebagai masukan bagi lapisan klasifikasi *fully connected* untuk memprediksi kelas sentimen. Konfigurasi model NN-BERT ditunjukkan pada Tabel 4.10.

Tabel 4.10: Konfigurasi model NN-BERT

Parameter	Nilai
<i>Encoder</i>	BERT, <i>frozen</i>
<i>Dimensi input</i>	768 ([CLS])
<i>Classifier</i>	FC, 1 lapisan
<i>Dimensi tersembunyi</i>	256
<i>Fungsi aktivasi</i>	ReLU
<i>Dimensi output</i>	2 kelas
<i>Fungsi loss</i>	<i>Cross-Entropy</i>
<i>Optimizer</i>	Adam
<i>Learning rate</i>	5×10^{-5}
<i>Batch size</i>	256
<i>Epoch</i>	10

4.4.3.2 BERTopic

BERTopic digunakan sebagai model pembanding pada tugas pendeteksian topik. Model ini menggabungkan representasi semantik berbasis BERT, reduksi dimensi menggunakan UMAP, serta proses klasterisasi dengan HDBSCAN untuk menghasilkan kelompok topik yang koheren. Ringkasan konfigurasi yang digunakan ditunjukkan pada Tabel 4.11.

Tabel 4.11: Konfigurasi model BERTopic

Komponen	Parameter	Nilai
<i>Embedding</i>	Model	bert-base-uncased
	Dimensi	768
UMAP	n_neighbors	15
	n_components	5
	min_dist	0,0
	Metrik	Kosinus
HDBSCAN	min_cluster_size	10
	min_samples	5
	Metrik	Euclidean
	prediction_data	True
Topik	Jumlah topik (<i>K</i>)	30, 50, 80
	Metode reduksi	nr_topics
	Representasi topik	c-TF-IDF

Untuk menjaga konsistensi dengan JST-DEC, jumlah topik yang dievaluasi pada BERTopic ditetapkan sebanyak 30, 50, dan 80 topik. Pengaturan ini dilakukan agar perbandingan kinerja kedua metode dilakukan pada skenario jumlah topik yang sama.

4.5 Analisis Kinerja Model

Bagian ini menyajikan hasil evaluasi model pada dua tahap, yaitu pra-pelatihan *autoencoder* dan *joint training*. Evaluasi dilakukan pada dataset IMDB dan Yelp.

4.5.1 Analisis Kinerja Pra-pelatihan

Kinerja pra-pelatihan dievaluasi menggunakan nilai MSE pada data latih dan data validasi. Hasil pra-pelatihan pada dataset IMDB dan Yelp disajikan pada Tabel 4.12 dan Tabel 4.13.

Tabel 4.12: Perkembangan *loss* pra-pelatihan pada dataset IMDB

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	...	...
5	...	...
10	...	...
15	...	...
20	...	...
25	...	...
30	...	...

Tabel 4.13: Perkembangan *loss* pra-pelatihan pada dataset Yelp

Epoch	<i>Train Loss</i>	<i>Validation Loss</i>
1	...	...
5	...	...
10	...	...
15	...	...
20	...	...
25	...	...
30	...	...

4.5.2 Analisis Kinerja *Joint Training*

Kinerja *joint training* dievaluasi berdasarkan dua kelompok metrik. Tugas analisis sentimen dievaluasi menggunakan *accuracy*, F1-score, *precision*, dan *recall*. Tugas pendeteksian topik dievaluasi menggunakan TC-NPMI dan TD.

4.5.2.1 Hasil Evaluasi pada Dataset IMDB

Hasil evaluasi pada dataset IMDB disajikan pada Tabel 4.14. Tanda (—) menunjukkan bahwa metrik tersebut tidak berlaku untuk model yang bersangkutan.

Tabel 4.14: Hasil evaluasi model pada dataset IMDB

Model	<i>K</i>	<i>Accuracy</i>	<i>F1</i>	<i>Precision</i>	<i>Recall</i>	TC-NPMI	TD
JST-DEC	30	...	...	...	...	...	...
	50	...	...	...	...	...	...
	80	...	...	...	...	...	...
BERTopic	30		—			...	...
	50		—			...	...
	80		—			...	...
NN-BERT	—	...	...	...	...	—	

4.5.2.2 Hasil Evaluasi pada Dataset Yelp

Hasil evaluasi pada dataset Yelp disajikan pada Tabel 4.15. Format tabel dibuat sama dengan dataset IMDB agar hasil kedua dataset dapat dibandingkan secara langsung.

Tabel 4.15: Hasil evaluasi model pada dataset Yelp

Model	<i>K</i>	<i>Accuracy</i>	<i>F1</i>	<i>Precision</i>	<i>Recall</i>	TC-NPMI	TD
JST-DEC	30	...	...	...	...	...	...
	50	...	...	...	...	...	...
	80	...	...	...	...	...	...
BERTopic	30		—			...	...
	50		—			...	...
	80		—			...	...
NN-BERT	—	...	...	...	...	—	

BAB 5

KESIMPULAN

@todo

Tambahkan kesimpulan dan saran terkait dengan pekerjaan yang dilakukan.

5.1 Kesimpulan

5.2 Saran

DAFTAR PUSTAKA

- [1] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Cham: Springer, 2018. ISBN: 978-3-319-94462-3. DOI: 10.1007/978-3-319-94463-0.
- [2] James Allan. *Topic Detection and Tracking: Event-Based Information Organization*. Vol. 12. The Information Retrieval Series. New York: Springer, 2002. ISBN: 978-0-7923-7664-4.
- [3] Dimo Angelov. “Top2Vec: Distributed Representations of Topics”. *arXiv preprint arXiv:2008.09470* (2020).
- [4] Jonathan Baxter. “A Model of Inductive Bias Learning”. *Journal of Artificial Intelligence Research* 12 (2000), pp. 149–198. DOI: 10.1613/jair.731.
- [5] James C. Bezdek. *Pattern Recognition with Fuzzy Objective Function Algorithms*. New York: Springer Science & Business Media, 1984. ISBN: 978-1-4757-0450-1. DOI: 10.1007/978-1-4757-0450-1.
- [6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [7] David M. Blei. “Probabilistic topic models”. *Communications of the ACM* 55.4 (2012), pp. 77–84.
- [8] David M. Blei, Andrew Y. Ng, dan Michael I. Jordan. “Latent Dirichlet allocation”. *Journal of Machine Learning Research* 3 (2003), pp. 993–1022.
- [9] Jordan Boyd-Graber, Yuening Hu, dan David Mimno. “Applications of topic models”. *Foundations and Trends in Information Retrieval* 11.2–3 (2017), pp. 143–296.
- [10] Ricardo J. G. B. Campello, Davoud Moulavi, dan Jörg Sander. “Density-Based Clustering Based on Hierarchical Density Estimates”. *Advances in Knowledge Discovery and Data Mining* (2013), pp. 160–172.
- [11] Dallas Card, Chenhao Tan, dan Noah A. Smith. “A Neural Framework for Generalized Topic Models”. *arXiv preprint arXiv:1705.09296* (2017).
- [12] Rich Caruana. “Multitask Learning”. *Machine Learning* 28 (1997), pp. 41–75. DOI: 10.1023/A:1007379606734.

- [13] Huimin Chen *et al.* “Neural Sentiment Classification with User and Product Attention”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 1650–1859.
- [14] Kyunghyun Cho *et al.* “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734.
- [15] Junyoung Chung *et al.* “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling”. *arXiv preprint arXiv:1412.3555* (2014).
- [16] Ronan Collobert *et al.* “Natural Language Processing (Almost) from Scratch”. *Journal of Machine Learning Research* 12 (2011), pp. 2493–2537.
- [17] Jacob Devlin *et al.* “BERT: Pre-training of deep bidirectional transformers for language understanding”. *arXiv preprint arXiv:1810.04805* (2018).
- [18] Adji B. Dieng, Francisco J. R. Ruiz, dan David M. Blei. “Topic Modeling in Embedding Spaces”. *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 439–453.
- [19] Long Duong *et al.* “Low Resource Dependency Parsing: Cross-lingual Parameter Sharing in a Neural Network Parser” (2015), pp. 845–850.
- [20] Martin Ester *et al.* “A Density-based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. 1996, pp. 226–231.
- [21] Ian Goodfellow, Yoshua Bengio, dan Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 978-0-262-03561-3. URL: <http://www.deeplearningbook.org>.
- [22] Theresia Gowandi, Hendri Murfi, dan Siti Nurrohmah. “Performance Analysis of Hybrid Architectures of Deep Learning for Indonesian Sentiment Analysis”. *Soft Computing in Data Science: 6th International Conference, SCDS 2021, Proceedings*. Ed. by Azlinah Mohamed *et al.* Vol. 1489. Communications in Computer and Information Science. Springer, 2021, pp. 18–27. DOI: 10.1007/978-981-16-7334-4_2.
- [23] Alex Graves. “Generating Sequences with Recurrent Neural Networks”. *arXiv preprint arXiv:1308.0850* (2013).
- [24] Maarten Grootendorst. “BERTopic: Neural topic modeling with a class-based TF-IDF procedure”. *arXiv preprint arXiv:2203.05794* (2022).

- [25] Lin Gui *et al.* “Multi-Task Mutual Learning for Joint Sentiment Classification and Topic Detection”. *IEEE Transactions on Knowledge and Data Engineering* 34.4 (2022), pp. 1915–1927.
- [26] Xifeng Guo *et al.* “Improved Deep Embedded Clustering with Local Structure Preservation”. *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*. 2017, pp. 1753–1759. DOI: 10.24963/ijcai.2017/243. URL: <https://www.ijcai.org/proceedings/2017/0243.pdf>.
- [27] Dan Hendrycks dan Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. *arXiv preprint arXiv:1606.08415* (2016). URL: <https://arxiv.org/abs/1606.08415>.
- [28] Geoffrey E. Hinton dan Ruslan R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks”. *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647.
- [29] Yohan Jo dan Alice H. Oh. “Aspect and Sentiment Unification Model for Online Review Analysis”. *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*. 2011, pp. 815–824.
- [30] Yoon Kim. “Convolutional Neural Networks for Sentence Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1746–1751.
- [31] Gary King dan Langche Zeng. “Logistic Regression in Rare Events Data”. *Political Analysis* 9.2 (2001), pp. 137–163.
- [32] Diederik P. Kingma dan Jimmy Ba. “Adam: A Method for Stochastic Optimization”. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. Published as a conference paper at ICLR 2015. 2015. URL: <https://arxiv.org/abs/1412.6980>.
- [33] Diederik P. Kingma dan Max Welling. “Auto-Encoding Variational Bayes”. *arXiv preprint arXiv:1312.6114* (2013).
- [34] Daniel D. Lee dan H. Sebastian Seung. “Learning the Parts of Objects by Non-Negative Matrix Factorization”. *Nature* 401.6755 (1999), pp. 788–791. DOI: 10.1038/44565.
- [35] Jiwei Li *et al.* “Visualizing and Understanding Neural Models in NLP”. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2016, pp. 681–691.

- [36] Qianqian Liang *et al.* “JST-RR model: Joint modeling of ratings and reviews in sentiment-topic prediction”. *arXiv preprint arXiv:2102.11048* (2021).
- [37] Chong Lin dan Yulan He. “Joint sentiment/topic model for sentiment analysis”. *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 2009, pp. 375–384.
- [38] Bing Liu. *Sentiment Analysis and Opinion Mining*. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers, 2012. ISBN: 978-1-60845-884-4.
- [39] Pengfei Liu, Xipeng Qiu, dan Xuanjing Huang. “Adversarial Multi-task Learning for Text Classification”. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. 2017, pp. 1–10.
- [40] Pengfei Liu, Xipeng Qiu, dan Xuanjing Huang. “Recurrent Neural Network for Text Classification with Multi-Task Learning”. *Proceedings of the 25th International Joint Conference on Artificial Intelligence*. 2016, pp. 2873–2879.
- [41] Andrew L. Maas *et al.* “Learning Word Vectors for Sentiment Analysis”. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. 2011, pp. 142–150.
- [42] Andrew L. Maas *et al.* “Learning Word Vectors for Sentiment Analysis”. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2011, pp. 142–150.
- [43] James MacQueen. “Some Methods for Classification and Analysis of Multivariate Observations”. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1. Berkeley, CA: University of California Press, 1967, pp. 281–297.
- [44] Pradipta Maji dan Sankar K. Pal. “Rough Set Based Generalized Fuzzy C-Means Algorithm and Quantitative Indices”. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 37.6 (2007). Cited as Maji *et al.*, 2002 for soft/hard clustering concepts, pp. 1529–1540. DOI: 10.1109/TSMCB.2007.906578.
- [45] Julian McAuley *et al.* “Image-based Recommendations on Styles and Substitutes”. *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2015, pp. 43–52.
- [46] Jon D. McAuliffe dan David M. Blei. “Supervised topic models”. *Advances in Neural Information Processing Systems*. 2008, pp. 121–128.

- [47] Qiaozhu Mei *et al.* “Topic Sentiment Mixture: Modeling Facets and Opinions in Weblogs”. *Proceedings of the 16th International Conference on World Wide Web*. 2007, pp. 171–180.
- [48] Yishu Miao, Lei Yu, dan Phil Blunsom. “Neural Variational Inference for Text Processing”. *Proceedings of the International Conference on Machine Learning*. 2016, pp. 1727–1736.
- [49] Tomas Mikolov, Kai Chen *et al.* “Efficient estimation of word representations in vector space”. *arXiv preprint arXiv:1301.3781* (2013).
- [50] Tomas Mikolov, Ilya Sutskever *et al.* “Distributed Representations of Words and Phrases and Their Compositionality”. *Proceedings of Advances in Neural Information Processing Systems*. 2013, pp. 3111–3119.
- [51] Shruti Pandit dan Shikha Gupta. “A Comparative Study on Distance Measuring Approaches for Clustering”. *International Journal of Research in Computer Science* 2.1 (2011), pp. 29–31. DOI: 10.7815/ijorcs.21.2011.011.
- [52] Bo Pang, Lillian Lee, dan Shivakumar Vaithyanathan. “Thumbs up? Sentiment Classification using Machine Learning Techniques”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2002, pp. 79–86.
- [53] Michael Röder, Andreas Both, dan Alexander Hinneburg. “Exploring the Space of Topic Coherence Measures”. *Proceedings of the 8th ACM International Conference on Web Search and Data Mining*. 2015, pp. 399–408.
- [54] Sebastian Ruder. “An Overview of Multi-Task Learning in Deep Neural Networks”. *arXiv preprint arXiv:1706.05098* (2017). URL: <https://arxiv.org/abs/1706.05098>.
- [55] David E. Rumelhart, Geoffrey E. Hinton, dan Ronald J. Williams. “Learning representations by back-propagating errors”. *Nature* 323 (1986), pp. 533–536.
- [56] Gerard Salton dan Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. *Information Processing & Management* 24.5 (1988), pp. 513–523.
- [57] Ivan Nunes da Silva *et al.* *Artificial Neural Networks: A Practical Course*. Cham: Springer, 2017. ISBN: 978-3-319-43162-8. DOI: 10.1007/978-3-319-43162-8.

- [58] Akash Srivastava dan Charles Sutton. “Autoencoding Variational Inference for Topic Models”. *Proceedings of the International Conference on Learning Representations*. 2017.
- [59] Chang Sun *et al.* “Fine-tuning BERT for joint entity and relation extraction in Chinese medical text”. *Proceedings of the 12th International Conference on Knowledge Science, Engineering and Management*. 2019, pp. 197–207.
- [60] Mukund Sundararajan, Ankur Taly, dan Qiqi Yan. “Axiomatic Attribution for Deep Networks”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 3319–3328.
- [61] Duyu Tang, Bing Qin, dan Ting Liu. “Aspect Level Sentiment Classification with Deep Memory Network”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 214–224.
- [62] Duyu Tang, Bing Qin, dan Ting Liu. “Document Modeling with Gated Recurrent Neural Network for Sentiment Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2015, pp. 1422–1432.
- [63] Ashish Vaswani *et al.* “Attention Is All You Need”. *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [64] Pascal Vincent *et al.* “Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion”. *Journal of Machine Learning Research* 11 (2010), pp. 3371–3408.
- [65] Sida Wang dan Christopher D. Manning. “Baselines and Bigrams: Simple, Good Sentiment and Topic Classification”. *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics*. 2012, pp. 90–94.
- [66] Junyuan Xie, Ross Girshick, dan Ali Farhadi. “Unsupervised deep embedding for clustering analysis”. *International Conference on Machine Learning (ICML)*. 2016.
- [67] Zichao Yang *et al.* “Hierarchical Attention Networks for Document Classification”. *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, pp. 1480–1489.
- [68] Hendi Yusdiansyah dan Kuspriyanto Suryadi. “Centroid-Based Clustering for Text Classification”. *Journal of Physics: Conference Series* 1192.1 (2019), p. 012022. DOI: 10.1088/1742-6596/1192/1/012022.

- [69] Matthew D. Zeiler dan Rob Fergus. “Visualizing and Understanding Convolutional Networks”. *European Conference on Computer Vision (ECCV)*. Springer, 2014, pp. 818–833.
- [70] Aston Zhang *et al.* *Dive into Deep Learning*. Cambridge University Press, 2023. ISBN: 978-1-009-38943-3. URL: <https://d2l.ai>.
- [71] Xiang Zhang, Junbo Zhao, dan Yann LeCun. “Character-level Convolutional Networks for Text Classification”. *Advances in Neural Information Processing Systems*. 2015, pp. 649–657.
- [72] Xiang Zhang, Junbo Zhao, dan Yann LeCun. “Character-level Convolutional Networks for Text Classification”. *Advances in Neural Information Processing Systems*. Vol. 28. Curran Associates, Inc., 2015.
- [73] Yinhan Zhang, Hao Zhao, dan Ming Li. “Neural topic models: A survey”. *arXiv preprint arXiv:2004.11906* (2020).
- [74] Yongliang Zhang *et al.* “Multi-task mutual learning for joint sentiment classification and topic detection”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), pp. 2407–2417.
- [75] Yu Zhang dan Qiang Yang. “A Survey on Multi-Task Learning”. *arXiv preprint arXiv:1707.08114* (2017).
- [76] Zhibin Zhou *et al.* “A comprehensive survey on clustering: From traditional methods to modern deep learning algorithms”. *arXiv preprint arXiv:2102.08012* (2021).